University of Science & Technology

Faculty of Engineering

Department of Electronic Systems and Computer Engineering

Forest Fire Detection System

Submitted by:
- Aabda Faiez Ahmed Mohamed
- Amro Mohamed Ahmed Elshikh
- Mohamed Gassim Amin Ahmed
- Yousif Bakhet Saad Aljamal

Supervised by:
T. Husameldin Babikir Elnour Osman

June-2026

قال الله تعالى:

﴿ يَرْفَعِ اللَّهُ الَّذِينَ آمَنُوا مِنْكُمْ وَالَّذِينَ أُوتُوا الْعِلْمَ دَرَجَاتٍ وَاللَّهُ بِمَا تَعْمَلُونَ خَبِيرٌ ﴾

سورة المجادلة: الآية [11]

الإهداء

إلى من كان سندًا لي في هذا الطريق،

إلى من آمنوا بي حين ترددت، ودفعوني حين تعثرت،

إلى من زرعوا في داخلي معنى الاجتهاد والصبر والإصرار.

إلى أسرتي الكريمة، مصدر الحب والدعم الذي لا ينضب،

وإلى كل من كان له أثر جميل في رحلتي العلمية والحياتية،

أهدي هذا العمل المتواضع، راجية أن يكون ثمرة جهد يُشرفهم ويليق بهم.

الشكر والتقدير

بسم الله الرحمن الرحيم :

﴿وَمَا بِكُم مِّن نِّعْمَةٍ فَمِنَ اللَّهِ﴾ صدق الله العظيم.

الحمد لله، له الفضل في الأولىوالآخرة على ما أنعم به علينا من توفيق وإعانة، فله الحمد حتى يبلغ الحمد منتهاه.

إلى من علّمونا حروفًا من نور، وكلماتٍ من أثرٍ باقٍ، إلى من كانوا مشاعل علمٍ أنارت لنا دروب المعرفة، وصاغوا بعلمهم عقولنا، وغرسوا فينا حب الاجتهاد والسعي نحو النجاح، إلى أساتذتنا الأفاضل الذين حملوا أقدس رسالة، رسالة التعليم وبناء الأجيال، نتقدم لكم بجزيل الشكر وعظيم.الامتنان

ونخصّ بالتقدير والشكر **الأستاذ الفاضل: حسام الدين بابكر**، الذي لم يدّخر جهدًا في تقديم التوجيه والنصح، وكان لدعمه العلمي وأفكاره القيّمة الأثر الكبير في إثراء هذا البحث وإخراجه بالصورة المطلوبة. فقد كان نعم المعلم والموجه، فجزاه الله عنا خير الجزاء، وبارك له في علمه وعمله.

كما لا يفوتنا أن نشكر كل من علمنّا معنى التفاؤل والمضي قدمًا، وكل من كان له أثر في رحلتنا العلمية والإنسانية، فلكم جميعاً منا خالص التقدير والاحترام.

Abstract

Forest fires are considered one of the most severe environmental disasters, requiring early detection and effective monitoring systems to reduce their spread and minimize the resulting environmental and economic damage. This project aims to develop an intelligent system based on the Internet of Things (IoT) and an unmanned aerial vehicle (Drone) for early forest fire detection and real-time alerting. The proposed system integrates multiple sensors, including a temperature and humidity sensor (DHT22), a smoke and gas sensor (MQ-2), and a Raspberry Pi NoIR Camera v2, which enables visual monitoring even in low-light conditions. These components are connected and processed using a Raspberry Pi Model 4, which serves as the main processing unit of the system. Additionally, an MCP3008 analog-to-digital converter is used to interface analog sensor signals with the Raspberry Pi.

The system continuously collects and analyzes environmental data by combining sensor readings with visual information from the camera to improve detection accuracy and reduce false alarms. When fire indicators are detected, the system triggers a real-time alert by sending an email notification to the user, including an image of the site along with numerical sensor data such as temperature, humidity, and smoke levels.

Furthermore, the system supports a preliminary response mechanism by enabling a user-confirmed action for early-stage fire suppression using a drone-based fire extinguishing system. This project focuses on enhancing early fire detection, utilizing multi-sensor data fusion, and providing real-time information to support rapid decision-making, ultimately contributing to improved safety for both the environment and firefighting teams.

المستخلص

تُعد حرائق الغابات من أخطر الكوارث البيئية التي تتطلب أنظمة مراقبة مبكرة وفعالة للحد من انتشارها وتقليل الخسائر الناتجة عنها. يهدف هذا المشروع إلى تطوير نظام ذكي يعتمد على إنترنت الأشياء (IoT) وطائرة بدون طيار (Drone) للكشف المبكر عن حرائق الغابات وتوفير تنبيهات دقيقة في الزمن الحقيقي.

يعتمد النظام على دمج مجموعة من الحساسات تشمل حساس درجة الحرارة والرطوبة (DHT22)، وحساس الدخان والغاز (MQ-2)، بالإضافة إلى كاميرا Raspberry Pi NoIR v2 التي توفر إمكانية الرؤية في الظروف منخفضة الإضاءة. يتم توصيل هذه الحساسات ومعالجة بياناتها باستخدام لوحة Raspberry Pi Model 4 باعتبارها وحدة المعالجة الرئيسية للنظام، مع استخدام محول الإشارات (MCP3008) لقراءة البيانات التناظرية لبعض الحساسات.

يعمل النظام على جمع وتحليل البيانات البيئية بشكل مستمر، حيث يتم دمج القراءات من الحساسات المختلفة مع الصور الملتقطة من الكاميرا لتحديد احتمالية وجود حريق بدقة أعلى وتقليل الإنذارات الكاذبة. في حال اكتشاف مؤشرات لوجود حريق، يقوم النظام بإرسال تنبيه فوري إلى المستخدم عبر البريد الإلكتروني، يتضمن صورة من الموقع بالإضافة إلى القيم الرقمية للحساسات مثل درجة الحرارة، الرطوبة، ومستوى الدخان.

كما يهدف النظام إلى دعم الاستجابة الأولية للحريق من خلال توفير إمكانية اتخاذ إجراء إخماد مبدئي بعد تأكيد المستخدم، وذلك باستخدام آلية مدمجة على الطائرة بدون طيار. يركز هذا المشروع على تحسين الكشف المبكر، وتعزيز الاعتماد على أنظمة متعددة الحساسات، وتوفير بيانات دقيقة في الزمن الحقيقي، مما يساهم في تقليل المخاطر على البيئة وعلى فرق الإطفاء.

Table of Content

Table of Figures

Table of Tables

Table Of Abbreviation

IoT	Internet of Things
I/O	Input/Output
MQ-2	gas sensor
RGB	Red Green Blue
DHT22	Temperature and Humidity Sensor
MCP3008	10-Bit Analog-to-Digital Converter
CPU	central processing unit
ROM	Read Only Memory
RAM	Random Access Memory
TI	Texas Instruments
TMS	Texas Instruments MOS/ Microcomputer system
IBM	International Business Machines
EEPROM	Electrically Erasable Programmable Read-Only Memory
SoC	system on a chip
GPU	Graphics processing unit
HEVC	High Efficiency Video Coding
USB	Universal Serial Bus
PCIe	Peripheral Component Interconnect Express
NAS	Network Attached Storage
GPIO	General Purpose Input/Output
GND	Ground
LED	Light Emitting Diode
LCD	Liquid Crystal Displays
UART	Universal Asynchronous Receiver-Transmitter
GPS	Global Positioning System
PWM	Pulse Width Modulation
OOP	Object-Oriented Programming
CMOS	Complementary Metal-Oxide-Semiconductor
ISP	Image Signal Processor
PCB	Printed Circuit Board
CSI	Camera Serial Interface
MIPI	Mobile Industry Processor Interface
MOS	Metal Oxide Semiconductor

LPG	liquefied petroleum gas
ADC	analogue-to-digital converter
SPI	Serial Peripheral Interface
PPM	Parts Per Million
VCC	Voltage Common Collector
AI	artificial intelligence
DAS	data acquisition system
MCU	Microcontroller Unit
OTP	one-time programmable
RTOS	real-time operating system
SDL	Serial Data Line
SCL	Serial Clock Line
HMI	Human-Machine Interface
SAR	successive approximation register
CSV	Comma-Separated Values
SD	Secure Digital
OS	Operating System
MIME	Multipurpose Internet Mail Extensions
SMTP	Simple Mail Transfer Protocol
TLS	Transport Layer Security
HTTP	Hypertext Transfer Protocol
HTML	HyperText Markup Language
JPEG	Joint Photographic Experts Group
YOLO	You Only Look Once
WSGI	Web Server Gateway Interface
ARM	Advanced RISC Machines
GUP	Graphics Processing Unit
SSL	Secure Sockets Layer
RFID	Radio-Frequency Identification
MQTT	Message Queuing Telemetry Transport
UAV	Unmanned Aerial Vehicle
FLIR	Forward Looking Infrared

Chapter One
Introduction

1.1 General Overview:

Forest fires are considered among the most severe natural disasters, causing significant environmental and economic damage. Over time, they have evolved from isolated incidents into a recurring global crisis due to climate change and rising temperatures. The danger of these fires lies in their rapid spread, especially in remote areas that are difficult to access or continuously monitor [1].

Historically, forest fire management relied entirely on manual efforts using basic tools such as axes and shovels to create firebreaks that limit the spread of flames. With the advancement of technology during the industrial revolution, pumps and engine-powered systems were introduced, enabling more efficient water transportation, along with the construction of observation towers for early fire detection. By the mid-twentieth century, a significant shift occurred with the use of aircraft in firefighting operations, including aerial tankers and specialized rapid-response teams capable of reaching remote locations.

In the modern era, smart technologies have become essential in early fire detection. Satellite remote sensing data is useful for identifying active fires, evaluating burned areas, and assessing fire emissions [2]. while drones are deployed to monitor hazardous areas and capture real-time images. Additionally, fire retardant chemicals are used to control the spread of flames. More recently, artificial intelligence techniques have been introduced to analyze environmental data and predict fire behavior, enabling proactive decision-making.

Despite these advancements, there remains a need for cost-effective, real-time intelligent systems that combine accurate sensing with rapid response capabilities. Therefore, this project proposes the design of an IoT-based multi-sensor smart system mounted on a drone, integrating multiple sensors and a camera to detect, analyze, and validate fire incidents in real-time, while providing real-time alerts and enabling appropriate response actions.

1.2 Problem Statement

Forest fires represent a serious environmental challenge that requires early detection and rapid response to limit their spread. Despite the availability of various detection systems, most existing solutions rely on a single data source, such as temperature or smoke alone, which may lead to reduced accuracy and an increased likelihood of false alarms.

In addition, many of these systems do not provide continuous monitoring for remote areas, resulting in delayed detection of fires at their early stages. Furthermore, responding to fire incidents is often delayed due to the difficulty of accessing the affected locations, allowing the fire to spread further and cause greater damage.

Therefore, there is a need for an integrated system capable of combining multiple data sources, providing real-time monitoring, and delivering accurate and timely alerts, while supporting effective decision-making for early and efficient fire response.

1.3 Solution Statement

The field of forest fire detection has experienced a major transformation with the introduction of advanced tools and methodologies. Emphasis on precision, speed, and comprehensive coverage has led to the development of modern detection techniques. Satellite imaging has emerged as a powerful tool, providing real-time or near-real-time observations of large-scale landscapes. Enhanced spatial resolution and spectral capabilities of these satellites provide unprecedented levels of detail.

Ground-based sensors complement this aerial perspective and demonstrate improved capabilities in detecting changes in temperature, infrared signatures, and smoke presence. These sensors, often networked together, form integrated monitoring systems that continuously observe forest environments. Automated camera systems further strengthened this network by continuously monitoring designated areas and, through AI integration, identifying potential threats in real time. Meanwhile, drones and aircraft have become increasingly prominent, offering flexible, wide-area monitoring while transmitting real-time data to support rapid intervention [2].

1.4 Objectives

- To enable early detection of forest fires using a multi-sensor-based system.
- To improve fire detection accuracy by integrating temperature, smoke, and visual data from a camera.
- To reduce false alarms by relying on multiple data sources (multi-sensor system).
- To send real-time alerts upon fire detection, including images and sensor data.
- To contribute to reducing risks to firefighters through early detection.
- To support decision-making by providing accurate and real-time information.

1.5 Thesis Layout

This thesis includes four chapters:

Chapter One: contains main idea of the project.

Chapter Two: given an overview literature review of project and describes the components that used to design the system.

Chapter Three: include the Methodology that will be used to create it.
Chapter Four: contains conclusion and recommendations for the future work and appropriate appendices are attached at the end of the project.

Chapter Two literature review

2.1 Background

A microcontroller is a computer-on-a-chip, or, it may be considered as a single-chip computer. Micro means a small device, and controller identifies it as a device to control objects, processes, or events. An alternate term used for it is embedded controller, because the microcontroller and its support circuits are often built into, or embedded in, the devices they control. Microcontrollers are usually used in devices from all aspect of life. Any device that measures, stores, controls, calculates, or displays information is a candidate for putting a microcontroller inside. The largest use for microcontrollers is in automobiles; just about every car manufactured today includes at least one microcontroller for engine control, and often more to control additional systems in the car. In desktop computers, you can find microcontrollers inside keyboards, modems, printers, and their peripherals [3].

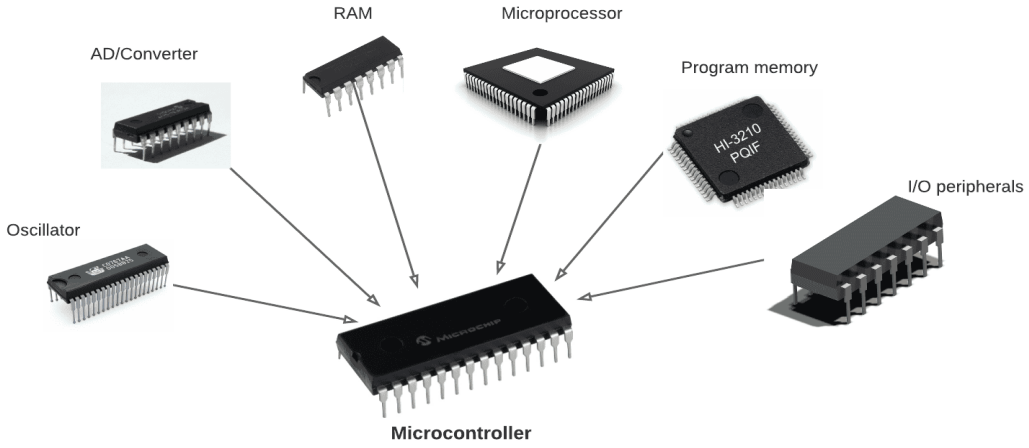


Fig 2.1: Microcontroller

Similarly, Consumer products that use microcontrollers include cameras, video recorders, compact disk players, and ovens etc. but these are just a few examples. A microcontroller is similar to the microprocessor inside a personal computer. To make a complete computer, a microprocessor requires memory for storing data and programs, and input/output (I/O) interfaces for connecting external devices like keyboards and displays. In contrast, a microcontroller is a single-chip computer because it contains memory and I/O interfaces in addition to the CPU. Because the amount of memory and interfaces that can fit on a single chip is limited, microcontrollers tend to be used in smaller systems. Examples of popular microcontrollers are Intel's 8052, Motorola's 68HC11, and Zilog's Z8. In 1971, the first microcontroller was invented by two engineers at Texas Instruments, according to the Smithsonian Institution. Gary Boone and Michael Cochran created the TMS 1000, which was a 4-bit microcontroller with built-in ROM and RAM. The microcontroller was used internally at TI in its calculator products from 1972 until 1974, and was refined over

the years. In 1974, TI offered the TMS 1000 for sale to the electronics industry. In addition, Intel also developed many important microcontrollers, two of which are the 8048 and 8051. The 8048 was one of Intel's first microcontrollers introduced in 1976 and was used as the processor in the IBM personal computer keyboard [7].

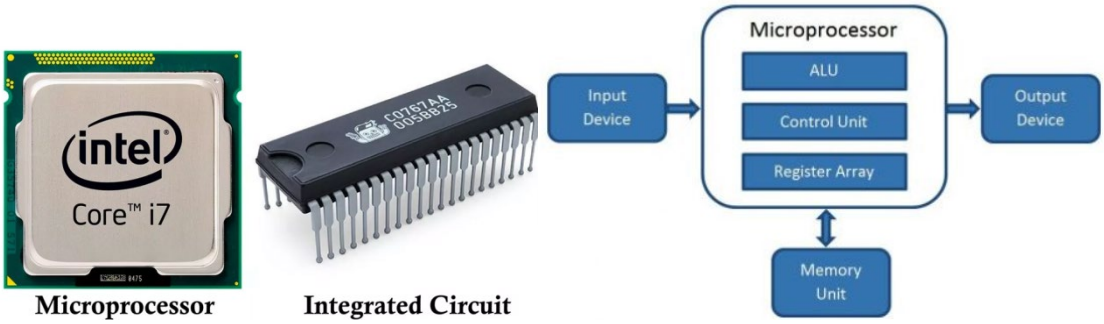


Fig 2.2: Microprocessor

The 8051 followed in 1980 and became one of the most popular microcontroller families. Variations of the 8051 architectures are still being produced today, making the 8051 one of the most long-lived electronics designs in history [8]. During the 1990s, microcontrollers with electrically erasable and programmable ROM (EEPROM) memories, such as flash memory, became available. These microcontrollers could be programmed, erased and reprogrammed using only electrical signals. Prior to the electrically reprogrammable devices, microcontrollers often required specialized programming and erasing hardware, which required that the device be removed from its circuit, slowing software development and making the effort more expensive. Overcoming this limitation, microcontrollers were able to be programmed and reprogrammed while in a circuit so devices with microcontrollers could be upgraded with new software without having to be returned to the manufacturer. Many current microcontrollers, such as those available from Microchip and Atmel, incorporate flash memory technology.

2.2 RASPBERRY PI 4 MODEL B

The Raspberry Pi 4 Model B represents a significant advancement in single-board computer (SBC) design, substantially expanding the capabilities of previous Raspberry Pi generations to encompass advanced computing, artificial intelligence inference at the edge, and industrial IoT gateway applications.

From an industrial and maker perspective, the Pi 4 occupies a unique position as a bridge between hobby electronics and production-grade embedded systems. Its broad software support spanning Raspberry Pi OS, Ubuntu, and various real-time operating system (RTOS) distributions combined with an active global community.

Originally conceived as an educational tool, the Pi 4 has transcended that initial purpose to become a legitimate platform for professional embedded system development, research, and prototyping.

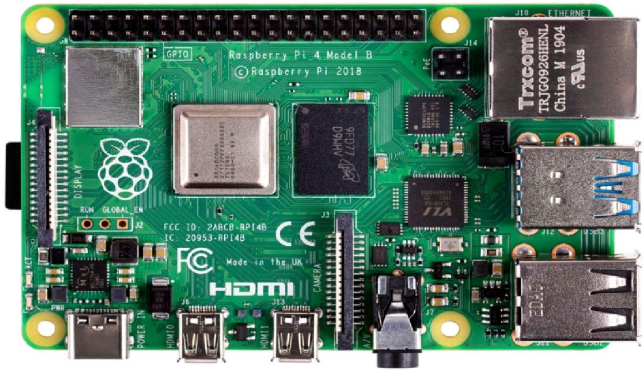


Fig 2.3: Raspberry Pi 4 Model B

2.2.1 Central Processing Unit Architecture

The computational core of the Raspberry Pi 4 is the Broadcom BCM2711 system-on-chip (SoC), which integrates a quad-core ARM Cortex-A72 (ARMv8-A, 64-bit) processor operating at 1.5 GHz, with later production revisions achieving 1.8 GHz. The transition from the ARM Cortex-A53 architecture used in the Raspberry Pi 3 to the Cortex-A72 in the Pi 4 represents an approximate 3× to 4× improvement in instructions-per-clock performance.

This performance improvement derives from the A72's out-of-order superscalar execution pipeline, which allows the processor to analyze a window of upcoming instructions and reorder their execution to avoid pipeline stalls caused by data dependencies or memory latency. In contrast, in-order processors including the Cortex-A53 execute instructions strictly in their programmed sequence, resulting in idle pipeline stages whenever a dependency creates a wait condition. For the computationally intensive tasks relevant to this project including image processing, neural network inference, and concurrent multi-threaded operation the out-of-order architecture of the A72 provides a meaningful practical advantage.

2.2.2 Memory and Peripheral Subsystems

The Raspberry Pi 4 introduced LPDDR4 SDRAM for the first time in the series, available in capacities of 1 GB, 2 GB, 4 GB, and 8 GB. LPDDR4 offers substantially higher memory bandwidth than the LPDDR2 used in earlier Pi models, which is critical for applications involving large data buffers such as uncompressed video frames or neural network weight tensors.

The graphics processing unit is a Video Core VI running at 500 MHz, supporting OpenGL ES 3.x and capable of hardware-accelerated decoding of 4K video at 60 frames per second using HEVC (H.265) encoding. Connectivity is substantially improved over previous generations: a dedicated Gigabit Ethernet controller no longer sharing bandwidth with the USB bus as in the Pi 3 provides true gigabit network performance.

Two USB 3.0 ports via an internal PCIe controller deliver high-speed external storage connectivity, while two Micro-HDMI ports support simultaneous 4K display output.

2.2.3 GPIO Interface and Communication Protocols

The Raspberry Pi 4 exposes a 40-pin GPIO header operating at 3.3V logic levels, providing the physical interface between the digital computing domain of the SoC and the analogue sensing and actuation domain of the physical world. These pins are software-configurable as digital inputs or outputs, and support several important serial communication protocols that are directly relevant to this project:

- SPI (Serial Peripheral Interface): A synchronous, full-duplex, master-slave protocol using four signal lines clock (SCLK), master-out slave-in (MOSI), master-in slave-out (MISO), and chip select (CS). SPI is used in this project for communication between the Raspberry Pi and the MCP3008 ADC. Its high clock speeds make it suitable for applications requiring rapid data acquisition.
- I2C (Inter-Integrated Circuit): A two-wire (SDA and SCL) multi-master, multi-slave protocol supporting up to 127 device addresses on a shared bus. I2C is commonly used for lower-bandwidth peripheral devices such as display modules and environmental sensors.
- UART (Universal Asynchronous Receiver-Transmitter): An asynchronous two-wire (TX and RX) serial protocol used for point-to-point communication with devices such as GPS modules, GSM modems, and serial debugging consoles.
- PWM (Pulse Width Modulation): A technique for simulating analogue output voltage by rapidly switching a digital pin between HIGH and LOW states, with the ratio of on-time to off-time (duty cycle) controlling the effective average voltage. The Pi 4 supports hardware PWM on specific pins (GPIO12, GPIO13, GPIO18, GPIO19) and software PWM on all GPIO pins.

Critical electrical constraints of the GPIO system must be respected to avoid permanent damage to the BCM2711 SoC. GPIO pins operate exclusively at 3.3V logic; applying 5V directly to a GPIO input will destroy the internal transistors. Individual GPIO pins are rated for a maximum output current of 16 mA, and the total current drawn from all GPIO pins simultaneously must not exceed 50 mA. Devices requiring higher drive currents — such as relay coils, motors, or high-brightness LEDs must be driven through transistor switching stages or dedicated driver integrated circuits

2.2.4 Suitability for the Forest Fire Detection System

The Raspberry Pi 4 was selected as the central processing platform for this project on the basis of three principal considerations. First, its computational performance is sufficient to run concurrent threads for camera capture, image processing or AI inference, sensor polling, web server operation, and email alert dispatch without resource contention. Second, its GPIO header provides native SPI bus access for the MCP3008 ADC and a single-wire digital input for the DHT22 sensor, eliminating the need for additional interface hardware. Third, its CSI-2 camera interface enables high-bandwidth, low-latency video capture from the Raspberry Pi camera module without consuming USB bandwidth or imposing USB protocol overhead.

2.3 RASPBERRY Pi Camera Module v1.3

The Raspberry Pi Camera Module v1.3, based on the OmniVision OV5647 CMOS image sensor, was selected as the visual sensing component of the fire detection system. Originally released in May 2013 through a technical collaboration between the Raspberry Pi Foundation and OmniVision Technologies, the module was designed to overcome the limitations of USB-connected cameras in embedded vision applications [4].

2.3.1 Design Motivation and Interface Architecture

Prior to the availability of dedicated camera modules for single-board computers, embedded vision systems relied on USB webcams, which introduced several significant academic and engineering limitations. USB cameras consume a disproportionate share of CPU processing cycles for data transfer, as the USB protocol imposes software-managed data movement between the camera and host memory. The USB 2.0 interface further constrains achievable frame rates and resolution combinations. Additionally, the latency introduced by USB protocol handshaking prevents use of USB cameras in timing-sensitive applications.

The Raspberry Pi Camera Module addresses these limitations by utilizing the MIPI CSI-2 (Mobile Industry Processor Interface Camera Serial Interface 2) standard, which allows raw image data to be transferred directly from the image sensor to the image signal processor (ISP) integrated within the Broadcom SoC via dedicated high-speed serial lanes. This architecture bypasses the CPU entirely for data movement, freeing processor resources for image processing and application logic. The physical connection uses a 15-pin flat flexible cable (FFC) carrying two high-speed data lanes and one clock lane, supporting data transfer rates of up to 1 Gbps per lane sufficient to sustain 1080p video at 30 fps or 720p at 60 fps.

2.3.2 Sensor Characteristics

The OmniVision OV5647 sensor is a 5-megapixel CMOS device fabricated using OmniBSI (back-side illuminated) technology. Back-side illumination repositions the photodetector layer to the back surface of the silicon wafer, away from the metal interconnect layers, increasing the proportion of incident light that reaches the photodetectors and improving sensitivity in low-light conditions. The sensor is paired with a fixed-focus lens with an aperture of f/2.9 and an effective focal length of approximately 3.6 mm.

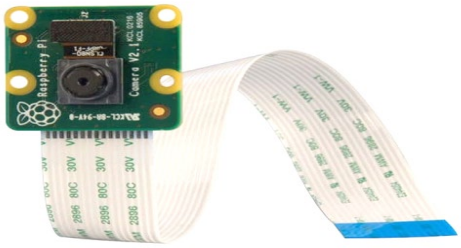


Fig 2.4: Raspberry Pi RGB Camera Module

The camera module includes an integrated IR cut filter in its standard RGB configuration, blocking wavelengths above 700 nm to prevent infrared light from contaminating color reproduction.

Raw image data from the sensor is delivered in Bayer pattern format to the SoC's integrated ISP, which performs demos icing, lens shading correction, automatic exposure control, and automatic white balance adjustment in hardware before presenting processed frames to software.

2.3.3 Relevance to Fire Detection

The camera module provides the visual sensing modality of the detection system the layer capable of directly observing flames and smoke as they appear in the field of view. Unlike the DHT22 and MQ135 sensors, which respond to secondary effects of fire (elevated temperature and airborne combustion by-products respectively), the camera provides direct visual confirmation of fire presence, making it the most interpretable modality for human operators reviewing alerts and saved snapshots. Its fixed focus is a limitation at distances significantly greater or less than approximately one meter, which represents a known constraint in the context of aerial deployment from drone altitude

2.4 MQ135 Smoke and Gas Sensor

The MQ135 is a metal oxide semiconductor (MOS) electrochemical gas sensor widely used in academic research and industrial applications for the detection of combustion by-

products, air quality monitoring, and early fire warning systems. Although the original chapter references the MQ-2 sensor, the MQ135 was selected for this implementation due to its broader sensitivity spectrum, which encompasses carbon dioxide (CO2), ammonia (NH3), nitrogen oxides (NOx), benzene, and smoke making it more appropriate for forest fire smoke detection than the LPG and methane-optimized MQ-2 [5].

Fig 2.5: MQ-2 sensor

2.4.1 Internal Structure and Operating Principle

The MQ135 sensor is an analogue transducer rather than a digital device. Its sensitive element is a layer of tin dioxide (SnO2), a metal oxide semiconductor material characterized by low electrical conductivity in clean air. The sensor module also incorporates an internal resistance heating element a coil of nichrome alloy that elevates the temperature of the SnO2 layer to approximately 200°C to 300°C, the range at which the surface chemical reactions that underpin the sensor's operating principle occur with sufficient speed and sensitivity. A double-layer stainless steel anti-explosion mesh surrounds the sensing element, preventing any sparks generated by the heater from igniting flammable gases in the surrounding environment.

The operating principle is based on the surface-effect chemisorption of oxygen. In clean air, oxygen molecules are adsorbed onto the SnO2 surface, capturing free electrons from the conduction band of the semiconductor and forming negatively charged oxygen ions. This electron depletion creates a potential barrier at grain boundaries within the SnO2 material, substantially increasing its electrical resistance. When reducing gases including smoke particles, carbon monoxide, and volatile organic compounds produced by combustion are present in the surrounding air, they react with the surface-adsorbed oxygen ions,

consuming them and releasing the previously captured electrons back into the conduction band. This electron restoration lowers the potential barrier and reduces the electrical resistance of the SnO2 layer in proportion to the concentration of the reducing gas.

The resulting change in resistance is converted to a voltage signal by a voltage divider circuit on the sensor module, in which the MQ135's variable resistance is placed in series with a fixed load resistor (typically 10 kΩ). The output voltage at the junction of this divider

the AOUT pin varies continuously between 0 V and the supply voltage (typically 5 V) as a function of gas concentration. This analogue output is the primary measurement signal used in Use Case 2 of this system.

2.4.2 Calibration Methodology

The MQ135 sensor requires careful calibration to produce quantitatively meaningful readings. The sensor's resistance in clean air (R_0) must first be established as a reference value, against which subsequent measurements are compared. The sensor must be allowed to warm up for a minimum of one to two minutes before initial use to allow the heater to reach its operating temperature, and manufacturer data sheets recommend a 24-to-48-hour burn-in period after first power-on for the SnO2 surface to fully condition.

For the purposes of this project, a baseline raw ADC value was established in clean ambient air after an appropriate warm-up period. Ten consecutive readings were recorded and averaged to produce a stable baseline of 3,798 raw units (on the 10-bit ADC scale of 0 to 65,535 as normalized by the Adafruit library). The smoke detection alert threshold was set at 130% of this baseline, corresponding to 4,937 raw units a value chosen to provide a margin above normal clean-air fluctuation while maintaining sensitivity to genuine smoke events.

2.4.3 Interface with the Raspberry Pi 4

Since the Raspberry Pi 4 provides only digital GPIO pins with no integrated ADC, the analogue output of the MQ135 cannot be read directly. Two interface strategies are available:

- Digital comparator output (D0): The MQ135 module incorporates an LM393 voltage comparator that compares the sensor output against a reference voltage set by an onboard potentiometer. When the sensor output exceeds the reference, the D0 pin transitions from HIGH to LOW, providing a simple threshold-crossing binary signal directly readable by a Raspberry Pi GPIO input pin. This approach is used in Use Case 1 of this system and requires no ADC hardware.

- Analogue ADC measurement (A0 via MCP3008): The continuous analogue output voltage is digitized by an external MCP3008 10-bit ADC connected to the Raspberry Pi's SPI bus. This approach provides a full 10-bit resolution measurement of gas concentration at each sampling interval, enabling quantitative logging, threshold adjustment in software, and trend analysis. This approach is used in Use Case 2 of this system.

2.5 DHT22 Temperature and Humidity Sensor

The DHT22 sensor, commercially designated AM2302, is a fully integrated digital environmental sensor combining capacitive humidity measurement and thermistor-based temperature measurement within a single compact package. It is classified academically as an integrated data acquisition system (DAS) because it incorporates not only the sensing elements but also an analogue-to-digital conversion stage and a digital communication interface within the same device, delivering pre-processed digital readings rather than raw analogue voltages [6].

2.5.1 Internal Architecture

The DHT22 comprises three functional subsystems that cooperate to convert physical environmental parameters into a digital data stream:

- Capacitive Humidity Sensor: A hygroscopic polymer film is interposed between two conductive electrodes, forming a capacitor whose capacitance varies as a function of the quantity of water vapor absorbed by the polymer. The dielectric constant of the polymer changes predictably with moisture content, producing a capacitance variation that the internal MCU converts to a relative humidity value expressed as a percentage.
- NTC Thermistor: A negative temperature coefficient resistor whose electrical resistance decreases with increasing temperature according to the Steinhart-Hart equation. The internal MCU measures this resistance and converts it to a temperature value using factory-programmed calibration coefficients stored in one-time programmable (OTP) memory.
- Internal Microprocessor Unit: An 8-bit microcontroller within the sensor package performs analogue-to-digital conversion of both sensing elements, applies the stored calibration corrections, and serializes the resulting values into the 40-bit data frame transmitted over the single-wire interface.

2.5.2 Single-Wire Communication Protocol

The DHT22 does not implement any standard serial protocol such as I2C or SPI. Instead, it uses a proprietary single-wire half-duplex interface in which a single bidirectional data line carries both the request signal from the host and the response data from the sensor. Each measurement cycle transmits a 40-bit data frame structured as follows:

- 16 bits of relative humidity data (integer and decimal parts, multiplied by 10 in the raw value)
- 16 bits of temperature data (integer and decimal parts, multiplied by 10 in the raw value, with the most significant bit indicating negative temperature)

- 8 bits of checksum (the sum of the four preceding bytes modulo 256) for error detection

The timing-sensitive nature of this protocol presents a practical challenge on Linux-based systems such as Raspberry Pi OS, which are non-real-time operating systems (Non-RTOS). The Linux kernel scheduler may preempt the GPIO-monitoring process during a measurement cycle, causing bit timing errors and failed reads.

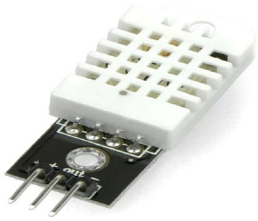


Fig 2.6: DHT22 sensor

The Adafruit Circuit Python DHT library mitigates this through interrupt-based bit detection and automatic retry logic. Failed readings which occur occasionally under normal operating conditions are handled gracefully by the sensor driver software, retaining the most recent valid reading until a new successful measurement is obtained.

2.5.3 Specifications and Operating Constraints

The DHT22 was selected over the DHT11 for this project on the basis of its substantially wider temperature range particularly its ability to measure temperatures down to -40°C its superior resolution of 0.1°C, and its greater accuracy of ±0.5°C.

Parameter	DHT22 (AM2302)	DHT11 (comparison)
Temperature range	-40°C to +80°C	0°C to +50°C
Temperature accuracy	±0.5°C	±2°C
Humidity range	0% to 100% RH	20% to 90% RH
Humidity accuracy	±2–5% RH	±5% RH
Resolution	0.1°C / 0.1% RH	1°C / 1% RH
Minimum sampling interval	2 seconds	1 second
Supply voltage	3.3V – 5.5V	3.3V – 5.5V
Communication	Single-wire proprietary	Single-wire proprietary
Internal calibration	Yes — OTP memory	Yes — OTP memory

Table 2.1: DHT22 specifications compared to DHT11, justifying DHT22 selection for this project.

For a fire detection system in which temperature elevation is a key alert trigger, measurement precision directly affects the reliability of the heat-based alert threshold.

2.5.4 Role in the Fire Detection System

The DHT22 contributes two distinct fire-related signals to the detection system. First, a sustained elevation of measured temperature above the configured threshold (default 45°C) directly indicates an abnormally hot environment consistent with fire proximity. Second, active combustion tends to reduce relative humidity in the surrounding air as the fire consumes moisture and heats the atmosphere. A significant rapid drop in humidity configured as a 20-percentage point decrease therefore serves as a supplementary fire indicator. Both conditions are evaluated independently in the sensor polling loop and contribute to the multi-sensor voting logic.

2.6 MCP3008 Analog-to-Digital Converter

The MCP3008 is an 8-channel, 10-bit successive approximation register (SAR) analog-to-digital converter manufactured by Microchip Technology. It serves as an indispensable interface component in this system, bridging the fundamental incompatibility between the analogue output of the MQ135 gas sensor and the exclusively digital GPIO interface of the Raspberry Pi 4 [7].

2.6.1 Internal Architecture

Unlike general-purpose processors, the MCP3008 does not incorporate a CPU in the conventional sense. Instead, it contains dedicated analogue signal processing circuitry optimized for the single task of converting an input voltage to a digital binary representation. Its principal internal subsystems are:

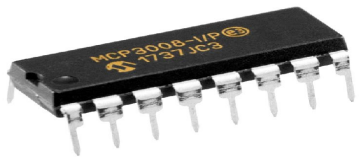


Fig 2.7: MCP3008

- Sample and Hold Circuit: A switched-capacitor circuit that captures and holds the input voltage at a precise instant in time, preventing the measured value from changing during the conversion process which takes a finite duration and thereby ensuring that the digital result accurately represents a specific instantaneous voltage rather than a time-averaged value.

- Successive Approximation Register (SAR): The conversion engine of the ADC. The SAR implements a binary search algorithm across the 10-bit digital range. Starting from the most significant bit, it sets each bit in sequence and compares the resulting DAC output voltage to the held input voltage. If the DAC voltage exceeds the input, the bit is cleared; otherwise, it is retained. After ten such comparisons, the register contains the closest 10-bit digital representation of the input voltage, with a conversion time of approximately 5 µs at maximum clock rate.
- Capacitive DAC (Digital-to-Analogue Converter): A capacitor array used internally by the SAR to generate the comparison reference voltage at each step of the binary search.
- SPI Interface Logic: A state machine managing the full-duplex SPI communication protocol receiving channel selection commands from the master device (Raspberry Pi) and transmitting the resulting 10-bit conversion result.

2.6.2 Technical Specifications

The MCP3008 analog-to-digital converter was selected as the interfacing component between the MQ135 analog gas sensor and the Raspberry Pi 4's digital GPIO interface, which natively lacks any analog input capability. Operating over a standard SPI bus, the MCP3008 provides a cost-effective and reliable solution for acquiring continuous analog voltage readings from the sensor and converting them into digital values that the Pi can process in software. Table 2.6.2 below summarizes the key technical specifications of the MCP3008 and highlights their direct relevance to the requirements of this air quality monitoring system.

Parameter	Specification	Relevance to System
Resolution	10-bit (1024 levels)	3.22 mV per LSB at 3.3V reference — sufficient precision for smoke concentration monitoring
Input channels	8 single-ended or 4 differentials	Channel 0 used for MQ135; channels 1–7 available for future sensor expansion
Sampling rate	Up to 200 ksps at 5V	Far exceeds the 0.5 Hz polling rate required by this application
Supply voltage	2.7V to 5.5V	Compatible with Pi's 3.3V supply rail without level shifting
Reference voltage	Equal to VDD (3.3V)	Sets full-scale input range; any fluctuation in 3.3V rail directly introduces measurement error

Communication	SPI (modes 0,0 and 1,1)	Native SPI bus support on Pi 4 via GPIO11 (SCLK), GPIO10 (MOSI), GPIO9 (MISO), GPIO8 (CE0)
Package	DIP-16	Compatible with standard 830-point breadboard
Current consumption	~500 µA during conversion	Negligible load on Pi's 3.3V supply rail

Table 2.2: MCP3008 technical specifications and their relevance to the detection system.

2.6.3 Voltage-to-Concentration Conversion

The 10-bit output of the MCP3008 represents the input voltage as an integer in the range 0 to 1023, where 0 corresponds to 0 V and 1023 corresponds to the reference voltage (3.3 V). The Adafruit MCP3xxx library normalizes this to a 16-bit range of 0 to 65,535 for convenience. The actual voltage can be recovered from the raw ADC value using the transfer function:

$$V_{out} = (ADC_{value} \times V_{ref}) / 65535 \quad \text{..... EQU 2.1}$$

where V_{ref} is 3.3 V. The gas concentration in the surrounding air is then proportional to the sensor resistance R_s, which is derived from V_{out} and the known load resistance R_L (typically 10 kΩ on the MQ135 module) according to:

$$R_s = R_L \times (V_{ref} - V_{out}) / V_{out} \quad \text{..... EQU 2.2}$$

For the threshold-based detection approach employed in this system, precise PPM calibration using the full logarithmic R_s / R_0 relationship was not required. Instead, a relative baseline comparison was used, in which the clean-air ADC value is established empirically and the alert threshold is set at a fixed multiple of that baseline. This approach is robust to sensor-to-sensor variation and eliminates the need for reference gas calibration.

2.6.4 Accuracy Considerations

Several factors affect the measurement accuracy of the MCP3008 in practice. Quantization error the inherent rounding error of the conversion process is ±0.5 LSB, corresponding to approximately ±1.6 mV at 3.3V reference. Supply voltage noise on the 3.3V rail of the Raspberry Pi directly manifests as measurement error, since the reference voltage equals the supply voltage; for precision applications, a dedicated low-noise precision voltage reference is recommended for the VREF pin. High source impedance at

the sensor output may prevent the sample-and-hold capacitor from fully charging within the available sampling time, causing the measured value to be lower than the true input voltage; this effect is negligible for the MQ135's output impedance in normal operation.

2.7 SYSTEM Integration and Related Work

The integration of multiple sensing modalities into a unified fire detection platform builds upon a body of literature examining multi-sensor environmental monitoring systems. The concept of sensor fusion combining the outputs of physically and chemically independent sensors to produce a more reliable composite assessment than any individual sensor could provide is well established in the embedded systems and safety engineering literature [8].

2.7.1 Multi-Sensor Fire Detection Approaches

Traditional fire detection systems in buildings rely on a single sensing modality typically ionization-based smoke detectors or fixed-temperature heat detectors. Research has consistently demonstrated that single-sensor systems exhibit unacceptably high false alarm rates in environments with airborne dust, cooking vapors, or normal temperature fluctuations, motivating the development of multi-sensor fusion approaches. Gottuk et al. demonstrated that combining photoelectric smoke sensing with carbon monoxide detection substantially reduced false alarm rates compared to either sensor alone. The same principle requiring agreement between multiple independent sensing channels before issuing an alert is applied in this project at the embedded system level [9].

The application of computer vision to fire detection has been an active research area since the early 2000s. Early approaches relied on color-based flame detection using HSV (Hue-Saturation-Value) color space analysis, exploiting the characteristic orange-red appearance of visible flames. More recent work has employed convolutional neural networks trained on labelled fire and smoke image datasets, achieving substantially higher detection rates and lower false positive rates than color-based methods, particularly under variable lighting conditions and at longer distances. The YOLOv8 nano model employed in Use Case 2 of this system represents the application of this state-of-the-art deep learning approach within the computational constraints of an embedded platform.

2.7.2 Edge Computing for Environmental Monitoring

The deployment of computational intelligence directly on embedded sensing platforms a paradigm known as edge computing or embedded AI avoids the latency, bandwidth cost, and connectivity dependency associated with cloud-based processing. For a time-critical application such as fire detection, in which a delay of even tens of seconds between fire ignition and alert dispatch could have significant consequences, edge processing is architecturally preferable to cloud-dependent approaches. The Raspberry Pi

4's ARM Cortex-A72 processor, while not equipped with a dedicated neural processing unit (NPU), is capable of running YOLOv8 Nano inference at 5 to 10 frames per second at 320-pixel input resolution sufficient for the temporal resolution requirements of this application [10].

2.7.3 Summary of Component Selection Rationale

The selection of each hardware component was driven by a careful evaluation of technical requirements, system compatibility, cost constraints, and the specific demands of an outdoor edge-deployed forest fire detection system. Priority was given to components that could operate reliably within the Raspberry Pi 4B's native interface ecosystem particularly SPI, CSI, and GPIO while minimizing processing overhead and avoiding unnecessary complexity in the hardware stack. Table 2.X below provides a consolidated summary of each selected component alongside the primary criteria that justified its inclusion and the alternatives that were considered and ultimately ruled out.

Component	Selected	Key Selection Criteria	Alternative Considered
Controller	Raspberry Pi 4B	Sufficient CPU for AI inference; native SPI/CSI; GPIO ecosystem	Arduino Mega (insufficient for image processing)
Camera	RPi Camera v1.3	CSI interface; no USB overhead; low cost; native picamera2 support	USB webcam (higher CPU load; latency)
Smoke sensor	MQ135	Broad gas sensitivity including CO2 and combustion VOCs	MQ-2 (optimized for LPG/methane, less suitable for forest smoke)
Temp/humidity	DHT22	0.1°C resolution; wide range; digital interface; pre-calibrated	DHT11 (insufficient resolution and range)
ADC	MCP3008	10-bit resolution; 8-channel SPI; 3.3V compatible; breadboard-friendly DIP package	ADS1115 (I2C, slower; overkill for this application)
AI model	YOLOv8 Nano	Smallest YOLO variant; detects fire and smoke; runs on Pi 4 CPU	YOLOv8s (too slow for real-time on Pi 4 without GPU)

Table 2.3: Component selection rationale and alternatives considered.

Chapter Three
Methodology

3.1 OVERVIEW

The system was conceived as a multi-sensor embedded detection platform built upon the Raspberry Pi 4 Model B single-board computer, integrating three independent and complementary sensing modalities: thermal and humidity monitoring via a DHT22 digital sensor, smoke and combustible gas concentration measurement via an MQ135 electrochemical sensor paired with an MCP3008 analog-to-digital converter, and visual fire detection via a Raspberry Pi Camera Module v1.3 processed using computer vision algorithms or an artificial intelligence object detection model, depending on the chosen implementation variant.

The methodology is structured around two distinct implementation use cases a cost-effective baseline system and an AI-enhanced high-accuracy system both sharing the same hardware platform but differing in the sophistication of their detection algorithms. A detailed description of the physical circuit construction, sensor wiring, software architecture, detection logic, alert mechanisms, and testing procedures follows in the sections below.

3.2 RESEARCH DESIGN AND SYSTEM PHILOSOPHY

The fundamental design philosophy of this system is the principle of multi-modal sensor fusion with a democratic voting mechanism. Rather than relying on any single sensing modality which would be susceptible to the false positive rates characteristic of individual sensor types the system requires agreement from at least two of the three sensing layers before issuing a confirmed fire alert. This approach was selected for the following reasons:

No single sensor technology is sufficient on its own for reliable fire detection in outdoor environments. Temperature sensors are susceptible to ambient heat from sunlight. Smoke sensors can be triggered by dust, humidity changes, or vehicle exhaust. Camera-based systems are sensitive to lighting conditions, shadows, and objects whose visual appearance resembles fire.

Multi-sensor agreement dramatically reduces the false positive rate, which is critical in a system intended for unattended autonomous deployment on an unmanned aerial vehicle over forested terrain.

The three sensing modalities are physically and chemically independent of one another, meaning that a fault or interference affecting one modality is unlikely to simultaneously affect the others, providing inherent redundancy.

The system was designed to be modular, with each sensing component encapsulated in its own software module, allowing independent testing, calibration, and replacement. The overall system architecture follows a producer-consumer concurrency model in which a fast camera capture thread produces annotated frames at approximately 20 frames per second for the live monitoring stream, while a slower sensor polling thread reads the DHT22 and MQ135 sensors at two-second intervals and evaluates the combined detection logic.

3.3 HARDWARE COMPONENTS AND SPECIFICATIONS

The following hardware components were selected, procured, and integrated into the detection device. Component selection was guided by criteria of cost-effectiveness, availability, compatibility with the Raspberry Pi GPIO ecosystem, and suitability for the intended outdoor deployment environment.

Component	**Model**	**Specification**	Role in System
Microcontroller	Raspberry Pi 4B	4-core ARM, 4GB RAM	Central processing unit runs all detection logic, web server, alert dispatch, and data logging
Temp/Humidity Sensor	DHT22 (AM2302)	±0.5°C, ±2–5% RH	Measures ambient temperature and humidity every two seconds to detect heat signatures and humidity drops associated with fire
Smoke/Gas Sensor	MQ135	CO2, NH3, NOx, benzene	Detects combustion by-products including smoke particles, carbon dioxide, and volatile organic compounds
ADC Converter	MCP3008	10-bit, 8-channel SPI	Converts the continuous analog voltage output of the MQ135 into a 10-bit digital value readable by the Pi over the SPI bus
Camera Module	RPi Camera v1.3	5MP OV5647, fixed focus	Captures live video frames for visual fire and smoke detection using image processing or AI inference
Breadboard	Standard 830-point	—	Prototyping platform for all sensor connections without soldering, enabling rapid reconfiguration during development
Power Supply	5V 3A USB-C	15W minimum	Powers the Raspberry Pi 4 and, through its 5V and 3.3V GPIO rails, all connected sensors

Table 3.1: Hardware components, specifications, and functional roles within the detection system.

3.4 PHYSICAL CIRCUIT CONSTRUCTION AND WIRING

This section describes in precise detail the physical electrical connections made between each sensor component and the Raspberry Pi 4 GPIO header. All wiring was performed with the Raspberry Pi fully powered down. The system uses the BCM (Broadcom) GPIO pin numbering scheme in software, while this section references physical board pin numbers for unambiguous identification. The physical pin layout of the 40-pin Raspberry Pi 4 header is as follows:

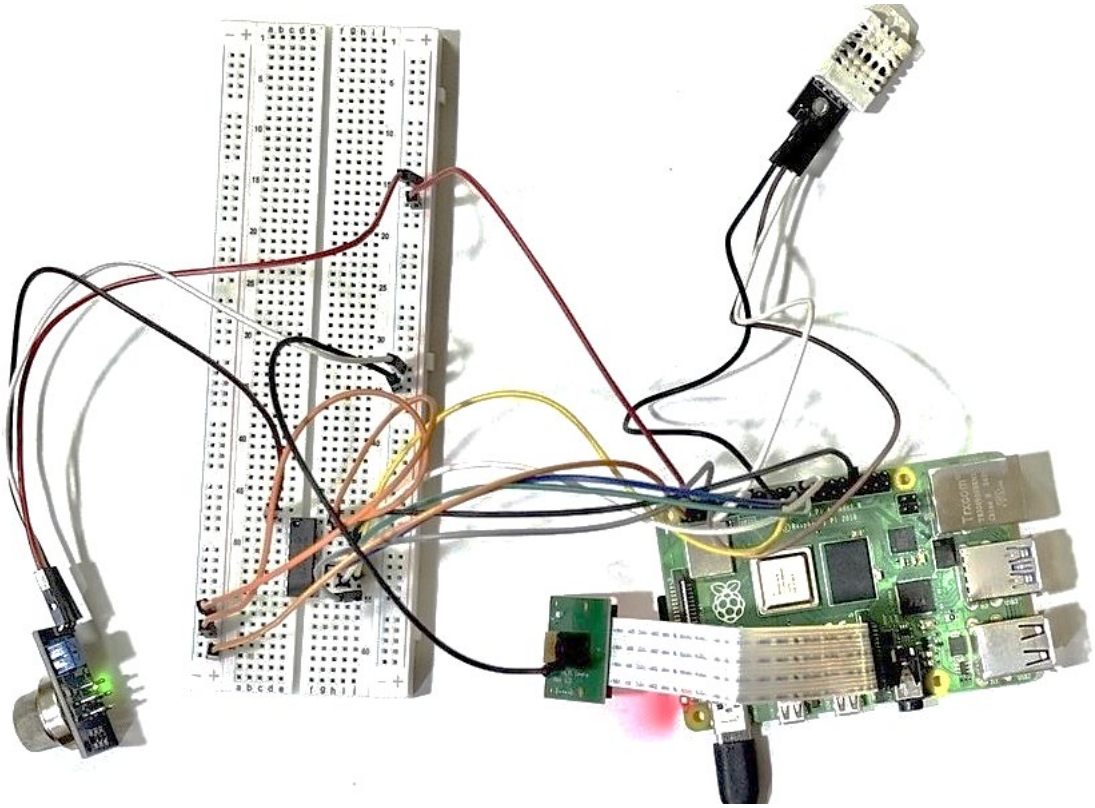


Fig1.0: physical wiring

Pin	Function	Pin	Function	Pin	Function
1	3.3V	2	5V	—	—
6	GND	7	GPIO4	19	GPIO10 / MOSI
21	GPIO9 / MISO	23	GPIO11 / SCLK	24	GPIO8 / CE0

Table 3.2: Relevant Raspberry Pi 4 GPIO pins used in this system.

3.4.1 DHT22 Temperature and Humidity Sensor

The DHT22 sensor communicates via a proprietary single-wire half-duplex digital protocol. The sensor was connected directly to the Raspberry Pi GPIO header using three jumper wires as follows:

Pin 1 (VCC) of the DHT22 connected to Raspberry Pi physical Pin 1 (3.3V power rail).
Pin 2 (DATA) of the DHT22 connected to Raspberry Pi physical Pin 7 (GPIO4 in BCM numbering). This is the single bidirectional data line through which the sensor transmits 40-bit humidity and temperature readings upon request.
Pin 4 (GND) of the DHT22 connected to Raspberry Pi physical Pin 6 (ground rail).

A10kΩ pull-up resistor between the DATA line and VCC is recommended by the manufacturer to ensure signal integrity on the single-wire bus. In the prototype implementation, the internal pull-up resistor of the Raspberry Pi GPIO was used as a practical alternative. The sensor was polled at two-second intervals, which is the minimum recommended polling rate for the DHT22 to allow the sensor's internal capacitor to discharge between measurements. Readings that returned None values which the DHT22 occasionally produces due to timing sensitivity were caught by exception handling and the previous valid reading was retained.

3.4.2 MQ135 Smoke and Gas Sensor with MCP3008 ADC

The Raspberry Pi 4 does not include an integrated analog-to-digital converter on its GPIO header, outputting only digital HIGH or LOW signals. Since the MQ135 sensor produces a continuous analog voltage output proportional to the concentration of detected gases, an MCP3008 8-channel 10-bit SPI ADC was introduced as an intermediary conversion stage. The SPI interface was enabled on the Raspberry Pi using the raspi-config system configuration utility prior to deployment, resulting in the appearance of the /dev/spidev0.0 and /dev/spidev0.1 device files confirming successful kernel module loading.

The MQ135 was connected as follows:
VCC of the MQ135 connected to Raspberry Pi physical Pin 2 (5V power rail). The MQ135 contains an internal heating element that requires 5V at approximately 150mA to reach its operating temperature of 200°C. Connecting to 3.3V would result in insufficient heating and inaccurate readings.
GND of the MQ135 connected to Raspberry Pi physical Pin 6 (ground rail).
Analog output (AOUT) of the MQ135 connected to Channel 0 (physical Pin 1) of the MCP3008 IC.

The MCP3008 ADC was wired to the Raspberry Pi SPI bus as detailed in the following table:

MCP3008 Pin	Pin Name	Pi Physical Pin	Description
1	CH0	— (MQ135 AOUT)	Analog input channel 0 receives the variable voltage from the MQ135 smoke sensor

9	DGND	Pin 6 (GND)	Digital ground reference for the SPI communication circuitry
10	CS / SHDN	Pin 24 (GPIO8 / CE0)	Chip Select pulled LOW by the Pi to activate the MCP3008 for SPI communication
11	DIN	Pin 19 (GPIO10 / MOSI)	Master Out Slave In carries command bytes from the Pi to the MCP3008 specifying which channel to sample
12	DOUT	Pin 21 (GPIO9 / MISO)	Master In Slave Out carries the 10-bit digitized result from the MCP3008 back to the Pi
13	CLK	Pin 23 (GPIO11 / SCLK)	SPI clock signal generated by the Pi at 1.35 MHz synchronizes data transfer between master and slave
14	AGND	Pin 6 (GND)	Analog ground separate from digital ground to minimize noise coupling into the analog measurement path
15	VREF	Pin 1 (3.3V)	Voltage reference for ADC sets the full-scale input range to 0–3.3V, giving a resolution of 3.3V ÷ 1024 ≈ 3.22 mV per LSB
16	VDD	Pin 1 (3.3V)	Supply voltage for the MCP3008 digital and analog circuitry must not exceed 3.3V when VREF = 3.3V

Table 3.3*:* MCP3008 ADC pin connections to the Raspberry Pi 4 SPI bus.

A critical practical note: the MCP3008 IC is physically symmetric in appearance and can be inserted into a breadboard in either orientation. During prototype construction, the chip was verified to be correctly oriented by locating the semicircular notch or dot marking on the package, which indicates the end nearest to Pin 1 (CH0). Incorrect chip orientation was identified during testing as a common failure mode producing consistently zero ADC readings.

3.4.3 Raspberry Pi Camera Module v1.3

The Raspberry Pi Camera Module v1.3, based on the OmniVision OV5647 5-megapixel sensor, was connected to the dedicated CSI-2 (Camera Serial Interface 2) port on the Raspberry Pi 4 board using the supplied 15-pin flat flexible ribbon cable. The connection procedure was as follows:

The Raspberry Pi was fully powered down and disconnected from the power supply before handling the camera connector.
The brown locking clip on the CSI connector on the Pi board was gently pulled upward to release the clamping mechanism.

The 15-pin ribbon cable was inserted fully into the connector slot with the exposed metal contacts facing toward the HDMI ports that is, with the blue plastic backing of the cable facing away from the HDMI ports.
The locking clip was pressed firmly downward to clamp the cable in position.
The same insertion procedure was repeated at the camera module end of the cable.

The camera module draws power directly through the CSI ribbon cable from the Pi board and requires no additional external power connections. Camera functionality was verified using the libcamera-hello command-line utility after enabling the camera interface through the system configuration and confirming the presence of the /dev/video0 device node. In software, the camera was accessed via the picamera2 Python library, which provides a high-level interface to the libcamera stack on modern Raspberry Pi OS (Bookworm).

3.5 SOFTWARE ARCHITECTURE AND IMPLEMENTATION

The system software was developed entirely in Python 3.13 and organized into a modular directory structure reflecting the separation of concerns between sensing, alerting, and presentation layers. This modular approach facilitates independent unit testing of each component, simplifies future maintenance and feature additions, and demonstrates software engineering principles appropriate to a graduation-level academic project.

3.5.1 Project Directory Structure

The software was organized into a modular directory structure in which each file has a single, clearly defined responsibility. This separation of concerns follows standard software engineering principles and was adopted deliberately to facilitate independent unit testing of each component, simplify future maintenance and feature additions. The complete module structure for both use cases is described in Table 3.4 below, with each module's role in the overall system outlined in the responsibility column.

File / Module	Responsibility
main.py	System entry point initializes all hardware interfaces, spawns concurrent threads, and launches the web server. Implements the combined fire detection voting logic.
config.py	Centralized configuration file containing all tunable parameters including email credentials, detection thresholds, GPIO pin assignments, and file system paths. Separating configuration from logic allows threshold adjustment without modifying detection code.
sensors/dht22.py	DHT22 sensor driver class wraps adafruit_dht library calls, handles read errors gracefully, compares readings against configured thresholds, and tracks humidity trends over time.
sensors/mq135.py	MQ135 sensor driver class interfaces with the MCP3008 over SPI via the adafruit_mcp3xxx library, returns raw 10-bit values and converted voltages, and evaluates smoke threshold conditions.

sensors/camera.py	Camera capture and fire detection class manages the picamera2 interface, runs the background calibration procedure, executes the frame-by-frame fire detection algorithm, and maintains the temporal sliding window for alert confirmation.
alerts/email_alert.py	Email notification module composes multi-part MIME email messages including plain text alert details and the attached snapshot image, and dispatches them via Gmail SMTP with TLS encryption.
web/stream.py	Flask web application defines the HTTP routes for the monitoring dashboard HTML page and the MJPEG live video stream endpoint, and renders sensor status data into the dashboard template.
snapshots/	Output directory receives timestamped JPEG images saved automatically whenever a confirmed fire event is detected. Images include visual annotations showing detected fire regions.
dataset/images/	(Use Case 2 only) stores raw camera frames from detection events to build a continuously growing training dataset for future model fine-tuning.
dataset/labels/	(Use Case 2 only) stores YOLO-format annotation label files corresponding to each image in the dataset, enabling supervised retraining.
models/fire_model.pt	(Use Case 2 only) pre-trained YOLOv8 Nano model weights file, loaded at startup for AI-based fire and smoke detection inference.

Table 3.4: Software module structure and responsibilities.

3.5.2 Concurrency Model

The system employs a multi-threaded concurrency model using Python's threading module to achieve smooth simultaneous video streaming and sensor monitoring. Two principal threads operate concurrently:

Camera Thread (fast loop, ~20 fps): Continuously captures frames from the camera, runs the fire detection algorithm on each frame, overlays visual annotations and sensor status text, and writes the annotated frame to a shared memory buffer protected by a threading. Lock. This thread operates at the maximum rate the camera and detection algorithm can sustain, typically 15 to 20 frames per second on the Raspberry Pi 4.
Sensor Thread (slow loop, 2-second interval): Reads the DHT22 and MQ135 sensors, evaluates all three alert conditions including the sliding window result from the camera thread, applies the voting logic to determine whether a fire event is confirmed, dispatches email alerts when appropriate, and updates the shared system status dictionary. The two-second polling interval matches the DHT22's minimum safe sampling rate.

The shared state consisting of the latest annotated frame and the system status dictionary is accessed under a mutual exclusion lock to prevent data races between the camera thread and the Flask web server thread that reads this state to serve client requests.

3.5.3 Key Python Libraries and Dependencies

The system software relies on a set of carefully selected third-party Python libraries, each chosen for its reliability, active maintenance, and compatibility with the Raspberry Pi 4 hardware and Raspberry PiOS Bookworm operating environment. All libraries were installed within a dedicated Python virtual environment to isolate the project dependencies from the system Python installation and ensure reproducible deployment. Table 3.5 below lists the principal libraries used, their installed versions as confirmed during development, and their specific role within the detection system.

Library	Version	Purpose
picamera2	0.3.34	High-level Python interface to the libcamera stack for camera capture on Raspberry Pi OS Bookworm
opencv-python	4.10	Image processing color space conversion, morphological operations, contour detection, frame annotation
ultralytics	8.4.33	YOLOv8 model loading, inference, and result post-processing (Use Case 2 only)
adafruit-dht	4.0.11	DHT22 sensor reading via Adafruit Circuit Python Blinka abstraction layer
adafruit-mcp3xxx	1.5.1	MCP3008 ADC SPI communication via Adafruit Circuit Python
flask	3.1.3	Lightweight WSGI web framework for the monitoring dashboard and MJPEG stream server
numpy	2.2.4	Numerical array operations for background modelling and pixel-level image processing
spidev	—	Low-level SPI bus access for MCP3008 communication verification during testing

Table 3.5: Python libraries used in the system software.

3.6 Implementation Use Cases

Two distinct implementation variants of the system were designed and evaluated. Both use cases share the same Raspberry Pi 4 hardware platform, DHT22 sensor, and overall software architecture, but differ significantly in their approach to camera-based fire detection and smoke concentration measurement. The two variants represent a deliberate trade-off spectrum between deployment cost, wiring complexity, computational requirements, and detection accuracy.

3.6.1 Use Case 1: Cost-Effective Baseline System

The first use case constitutes the simpler and more economically accessible implementation, intended for resource-constrained deployments or scenarios in which the primary objective is functional fire detection at minimal component cost.

3.6.1.1 Camera Detection: OpenCV Background Subtraction

In this variant, fire detection from the camera feed is performed entirely using classical computer vision techniques implemented with the OpenCV library, without any machine learning inference. The detection pipeline comprises the following stages:

Background Modelling: At system startup, 60 frames of the scene in its normal state without fire or smoke are captured at 50-millisecond intervals. The pixel-wise median of these 60 frames is computed to form a stable background reference image. The median operation was chosen in preference to the mean because it is robust to transient motion during the calibration period, since a pixel that moves for fewer than 30 of the 60 frames will not influence the median.

Difference Computation: For each incoming live frame, the luminance (greyscale) representation of the frame is subtracted from the calibrated background model on a pixel-by-pixel basis. Pixels that are substantially brighter than the corresponding background location are flagged as candidate foreground pixels. The brightness difference threshold is set dynamically during calibration to a value of max (80, background_mean × 0.5), ensuring the threshold adapts to the ambient lighting level of the deployment environment.

Morphological Filtering: The binary candidate mask is refined using morphological opening (erosion followed by dilation with a 5×5 kernel) to eliminate isolated noise pixels, followed by morphological dilation to close small gaps within genuine fire regions.

Contour Analysis: Connected regions of candidate pixels are extracted using contour detection. Only regions whose pixel area exceeds a configured minimum area threshold set to 200 pixels in the default configuration are classified as potential fire detections, discarding tiny noise artefacts.

Temporal Confirmation via Sliding Window: A sliding window of the last 20 frames is maintained. A camera fire alert is only raised when at least 4 of the 20 most recent frames contain a detected region above the area threshold. This temporal filtering requires the anomalous brightness to persist across multiple frames rather than appearing as a single-frame flash, reducing false positives from specular reflections, camera gain fluctuations, and other transient events.

3.6.1.2 Smoke Detection: Digital Threshold Output (D0)

In this simplified variant, only the digital output pin (D0) of the MQ135 sensor board is used, bypassing the need for the MCP3008 ADC entirely. The MQ135 module includes an onboard LM393 comparator that compares the sensor's analog output voltage against a reference voltage set by an onboard potentiometer. When the analog voltage exceeds this reference indicating a gas concentration above the manually configured threshold the D0 pin transitions from HIGH to LOW.

This digital output is connected directly to a Raspberry Pi GPIO input pin configured in pull-up mode. The software polls this pin and interpret a LOW state as a smoke alert condition. While this approach sacrifices the ability to obtain continuous quantitative gas concentration measurements, it eliminates the MCP3008 ADC from the circuit entirely, reducing the component count, wiring complexity, and cost of the system. The detection sensitivity is adjusted by physically turning the potentiometer on the MQ135 module.

3.6.1.3 Advantages and Limitations

Advantages are Reduced component count and wiring complexity: The ADC is eliminated, reducing the number of SPI connections and breadboard space required.

Lower cost: The MCP3008 ADC represents an additional cost that is not required in this variant.

No machine learning dependency: The system operates entirely with deterministic classical algorithms, requiring no model file download or inference engine.

Limitations are Binary smoke detection: The D0 output provides only threshold crossing information. No quantitative concentration data is available for logging, trending, or fine-grained alert grading.

Manual sensitivity adjustment: The smoke detection threshold must be physically adjusted via a screwdriver on the potentiometer, which is impractical in an autonomous aerial deployment.

Camera detection sensitivity to lighting: The background subtraction approach is sensitive to changes in ambient lighting. Sunlight direction changes, cloud cover transitions, and shadows can all produce false brightness differentials.

3.6.2 Use Case 2: AI-Enhanced High-Accuracy System

The second use case constitutes the more sophisticated and accurate implementation, incorporating a pre-trained deep learning object detection model for camera-based fire recognition and the full analog measurement capability of the MQ135 sensor via the MCP3008 ADC.

3.6.2.1 Camera Detection: YOLOv8 Nano AI Model

In this variant, fire and smoke detection from the camera feed is performed using a YOLOv8 Nano (You Only Look Once version 8, nano architecture variant) deep learning object detection model. YOLOv8 is a single-stage, anchor-free object detector developed by Ultralytics that performs detection, classification, and localization in a single forward pass through the neural network. The nano variant was selected because it represents the smallest

and fastest model in the YOLOv8 family, making it the most feasible option for real-time inference on the Raspberry Pi 4's ARM Cortex-A72 CPU without GPU acceleration.

The model used in this system was a pre-trained fire-and-smoke detection model obtained from an open-source repository, trained on a curated dataset of fire and smoke images using the standard YOLOv8 training pipeline. The model detects two object classes: class 0 (fire) and class 1 (smoke). The model weights file (best.pt, approximately 6MB) was loaded at system startup using the Ultralytics Python library.

The inference pipeline operates as follows:
Each camera frame is passed to the YOLO model's predict method with a confidence threshold of 0.35 and an inference image size of 320×320 pixels. The reduced inference resolution of 320 pixels (compared to the default 640 pixels) was chosen to approximately double inference speed at the cost of reduced detection sensitivity to very small distant objects.

The model returns a Results object containing bounding box coordinates, class indices, and confidence scores for each detected object.
Detections whose class corresponds to fire or smoke and whose confidence score exceeds the threshold are accepted. The bounding box coordinates are used to annotate the display frame and to crop the detected region for dataset collection.
The same 20-frame sliding window temporal confirmation mechanism used in Use Case 1 is applied: a camera alert is raised only when fire or smoke detections appear in at least 4 of the 20 most recent frames.

A significant additional capability in this variant is automatic dataset collection. Every frame in which a fire or smoke detection is confirmed is saved as a JPEG image to the dataset/images/ directory, and a corresponding YOLO-format label file is written to dataset/labels/. Each label file contains one line per detection in the format: class_index centre_x centre_y width height (all values normalized to the range [0, 1] relative to the image dimensions). This continuously growing labelled dataset can be used in future iterations to fine-tune the model on imagery captured from the actual deployment environment, demonstrating the concept of continuous learning and domain adaptation even if the retraining step itself is performed on more powerful computing hardware offline.

3.6.2.2 Smoke Detection: Analog Output via MCP3008

In this variant, the full analog output of the MQ135 sensor is read through the MCP3008 ADC over the SPI bus, providing a continuous 10-bit digital representation of the sensor's output voltage. The output value ranges from 0 to 65,535 in the Adafruit library's normalized scale, corresponding to 0V to 3.3V at the ADC input.

A baseline calibration procedure is performed before deployment. The system is powered on in a clean air environment and the MQ135 raw output is monitored for a stabilization period of several minutes until readings converge to a stable value. This stable value is recorded as the clean-air baseline (measured as approximately 3,798 raw units in the prototype). The smoke

detection alert threshold is then set at 130% of this baseline value (approximately 4,937 raw units), providing a margin above normal fluctuation while remaining sensitive to genuine smoke events.

This approach provides several advantages over the digital output used in Use Case 1: the continuous numeric reading allows the system to log concentration levels over time, provides a quantitative measure of smoke severity for graduated alert levels, and enables the detection threshold to be adjusted in software without physical hardware modification an important consideration for a remotely deployed autonomous system.

3.6.2.3 Advantages and Limitations

Advantages are Distance-robust fire detection: The YOLOv8 model was trained on diverse fire and smoke images and can detect fires at varying distances, sizes, orientations, and lighting conditions without background calibration.
Simultaneous fire and smoke classification: The AI model independently detects both fire (visible flames) and smoke (combustion by-products), providing two sub-categories of visual evidence within a single detection pass.
Quantitative smoke measurement: Continuous analog ADC readings enable concentration logging, trend analysis, and software-adjustable thresholds.
Automatic dataset growth: Every detection event contributes to the training dataset, creating a self-improving system over time.

Limitations are Higher computational load: YOLOv8 inference at 320-pixel resolution on the Pi 4 CPU introduces latency compared to the classical background subtraction approach, reducing effective detection frame rate.
Additional hardware required: The MCP3008 ADC, SPI wiring, and model weights file add complexity relative to Use Case 1.
pre-training domain gap: The model was trained on general fire and smoke imagery. Performance may differ in specific environments such as dense forest canopy views from drone altitude.

3.6.3 Comparative Summary

The two use cases represent distinct points on the trade-off spectrum between system simplicity and detection sophistication. Table 3.6 below provides a structured side-by-side comparison of the key technical characteristics, hardware requirements, and operational capabilities of each implementation variant, summarizing the differences discussed in the preceding sections and providing a concise reference for evaluating which approach is most appropriate for a given deployment scenario.

Feature	**Use Case 1 — Baseline**	**Use Case 2 — AI Enhanced**
Camera Detection Method	OpenCV background subtraction	YOLOv8 Nano deep learning model
Detects Fire	Yes — brightness anomaly	Yes — trained class 0

Detects Smoke	Partially — if optically dense	Yes — trained class 1
Smoke Sensor Output	Digital (D0) — threshold only	Analog (A0) via MCP3008 ADC
ADC Required	No	Yes — MCP3008
Background Calibration	Required at startup	Not required
Detection Range	Short to medium	Short to long
Lighting Sensitivity	High	Low
Dataset Collection	No	Yes — automatic
Inference Speed (Pi 4)	~20 fps	~5–10 fps at imgsz=320
Component Count	Lower	Higher
Estimated Cost	Lower	Slightly higher
Best Deployment	Budget indoor/outdoor fixed mount	Drone-mounted outdoor forest monitoring

Table 3.6: Feature comparison between the two implementation uses cases.

3.7 COMBINED FIRE DETECTION LOGIC

The combined fire alert logic is evaluated in the sensor polling thread every two seconds. The logic is designed to minimize false positives through multi-sensor agreement while maintaining sensitivity to genuine fire events. The evaluation proceeds as follows:

The heat alert condition is TRUE when the DHT22 temperature reading exceeds the configured TEMP_THRESHOLD (default: 45°C). This threshold was set conservatively above the expected maximum ambient temperature in the deployment environment to avoid triggering from natural daytime heating.

The smoke alert condition is TRUE when the MQ135 raw ADC reading (Use Case 2) or D0 digital pin state (Use Case 1) indicates gas concentration above the configured threshold.

The camera alert condition is TRUE when the sliding window count of fire/smoke detection frames meets or exceeds the HIT_THRESH value (default: 4 out of 20 frames).

A fire event is considered pending when the sum of active alerts across the three modalities equals or exceeds 2 (i.e., at least two of the three sensors agree).

A persistent alert counter is incremented by 1 for each polling cycle in which a pending condition exists, and decremented by 1 for each cycle in which it does not, bounded to the range [0, 20]. A fire event is confirmed when this counter reaches the ALERT_CONFIRM threshold (default: 3), requiring the multi-sensor agreement to persist across at least three consecutive polling cycles (approximately 6 seconds). This prevents a single momentary coincidence of two sensor readings from triggering an alert.

When a fire event is confirmed, the system saves an annotated snapshot image and dispatches an email alert. A cooldown period of 60 seconds is enforced between successive email dispatches to prevent notification flooding during sustained fire events while maintaining the visual alert on the dashboard.

3.8 ALERT AND NOTIFICATION SYSTEM

The alert and notification system is responsible for communicating confirmed fire detection events to the system operator. Upon confirmation of a fire event, the following sequence is executed:

The current annotated video frame is saved to the snapshots/ directory as a timestamped JPEG file. The annotation includes red bounding boxes around detected fire regions, the current temperature and smoke readings, and the individual alert status of each sensor.
An email message is composed using Python's smtplib and email. mime libraries. The message is a multi-part MIME email containing a plain-text body with the timestamp, detected sensor readings, and reason string identifying which sensors triggered the alert, plus the snapshot image attached as a JPEG file.
The email is dispatched to the configured recipient address via Gmail's SMTP server on port 465 with SSL/TLS encryption, authenticated using a Gmail application-specific password generated from the Google Account security settings.
Email dispatch is performed in a separate daemon thread to avoid blocking the sensor polling loop during the network operation.
The web dashboard status indicator transitions from green to red and displays the alert message to all connected monitoring clients. The dashboard auto-refreshes every three seconds.

3.9 LIVE MONITORING WEB DASHBOARD

Web-based monitoring interface was implemented using the Flask Python microframework. The dashboard is served on port 5000 of the Raspberry Pi and is accessible from any device on the same local area network by navigating to http://raspberrypi.local:5000 in a web browser. No internet connection is required for local monitoring.

The dashboard provides the following real-time information:
A live annotated MJPEG (Motion JPEG) video stream from the camera, delivered as a multipart HTTP response with each frame encoded as a JPEG at quality 70. The stream operates at approximately 20 frames per second in normal operation.
Current temperature (°C) and relative humidity (%) readings from the DHT22 sensor.
Current smoke concentration raw value from the MQ135 sensor.
Individual alert status indicators for the heat sensor, smoke sensor, and camera detection, displayed in a six-cell grid with black/white color coding for alert and normal states respectively.
Overall system fire confirmation status as a large status banner, green for all clear and red for fire detected.
Current alert counter level and timestamp.

3.10 TESTING AND VALIDATION PROCEDURE

Testing was applied to verify the correct operation of each hardware component individually, the integrated multi-sensor system as a whole, and the alert and notification pipeline.

3.10.1 Individual Sensor Verification

Prior to integrated system testing, each hardware component was individually verified using standalone Python test scripts:

DHT22 verification: A short polling script confirmed the sensor returned valid temperature and humidity readings. The prototype measured 23.6°C and 52.3% relative humidity in the test environment, consistent with measured ambient conditions.
MCP3008 and MQ135 verification: A standalone SPI test confirmed communication with the ADC (reading 0 indicated a wiring fault; after correcting a misplaced chip select wire, readings of approximately 3,715 to 3,843 raw units were obtained in clean air, confirming correct operation).
Camera verification: A single-frame capture script saved a test JPEG to disk, which was visually inspected to confirm correct focus, framing, and color rendering.
SPI bus verification: The presence of /dev/spidev0.0 and /dev/spidev0.1 device nodes was confirmed after enabling the SPI interface kernel module via raspi-config.

3.10.2 Sensor Calibration

The MQ135 sensor requires a warm-up period of 24 to 48 hours after initial power-on for the internal heating element to stabilize. The clean-air baseline was established after this warm-up period by recording ten consecutive readings over ten seconds and computing the mean, yielding a baseline of 3,798 raw units. The smoke detection threshold was set at 4,937 raw units (130% of baseline).

For Use Case 1 camera detection, the background calibration was performed by positioning the camera in its intended monitoring orientation with the scene in its normal state and allowing the 60-frame median computation to complete before testing. The computed difference threshold was verified to produce zero detections on the normal scene before proceeding to fire testing.

3.10.3 End-to-End Integrated Testing

End-to-end testing was performed using a standard butane lighter as a controlled fire source to simulate a small flame. The following test scenarios were executed and their outcomes recorded:

Test Scenario	Expected Outcome	Observed Outcome	Notes

Flame only — near camera	Camera alert only — no system alert	Pass	Single sensor alert — voting threshold not met
Smoke only — near MQ135	Smoke alert only — no system alert	Pass	Smoke threshold exceeded; camera and temperature normal
Flame and smoke — both sensors	System fire alert confirmed	Pass	Two sensor agreement; email dispatched; snapshot saved
Elevated temperature and smoke	System fire alert confirmed	Pass	DHT22 and MQ135 agreement; camera below threshold
All three sensors triggered	System fire alert confirmed	Pass	Maximum confidence — all three modalities active
Normal operation — no fire	No alert issued	Pass	System stable at baseline; dashboard showed all clear

Table 3.7: End-to-end test scenarios and outcomes.

3.10.4 Web Dashboard Verification

The live dashboard was verified on a mobile device connected to the same Wi-Fi network as the Raspberry Pi. The MJPEG stream was confirmed to display at a usable frame rate, sensor readings updated on each three-second page refresh, and the fire alert status correctly toggled between all-clear and fire-detected states during testing.

3.11 IoT Alert and Monitoring System

The Internet of Things (IoT) is a paradigm that extends internet connectivity beyond conventional computing devices laptops, smartphones, and servers to encompass a vast and growing ecosystem of physical objects embedded with sensors, actuators, microcontrollers, and wireless communication modules. These objects, ranging from industrial machinery and agricultural sensors to household appliances and wearable devices, are capable of collecting data from their physical environment, transmitting that data over the internet to cloud platforms or local servers, and receiving commands or configuration updates in return. The term was first coined by Kevin Ashton in 1999 in the context of supply chain management using RFID technology, but has since expanded to describe one of the most transformative technological developments of the twenty-first century.

3.11.1 IoT in Environmental and Fire Detection Applications

The application of IoT principles to environmental monitoring and early warning systems represents one of the most socially impactful uses of the technology. Traditional fire detection infrastructure comprising fixed smoke detectors, heat sensors, and manual patrol systems suffers from fundamental limitations in coverage area, response latency, and scalability. IoTenabled fire detection systems address these limitations by enabling dense sensor deployment over large geographic areas, real-time remote monitoring from any location,

automated alert dispatch within seconds of detection, and centralized data collection for pattern analysis and predictive modelling.

Academic research in this domain has demonstrated that IoT-based forest fire early warning systems can reduce the mean time from ignition to alert dispatch from hours the typical response time of patrol-based systems to seconds or minutes. The integration of multiple sensing modalities (temperature, smoke, and visual detection) with cloud-based alert infrastructure and mobile notification systems, as implemented in this project, represents the current state of practice in IoT-enabled environmental safety systems.

3.12 Blynk IoT Platform

Blynk is a commercial IoT platform founded in 2012 and launched publicly in 2014, designed to provide developers, researchers, and students with a comprehensive infrastructure for connecting embedded hardware to the internet and building mobile and web-based monitoring dashboards without requiring extensive backend development expertise. The platform abstracts the complexities of cloud infrastructure, database management, real-time data streaming, and mobile application development, allowing engineers to focus on the sensing and detection logic of their systems rather than the communication and presentation layers.

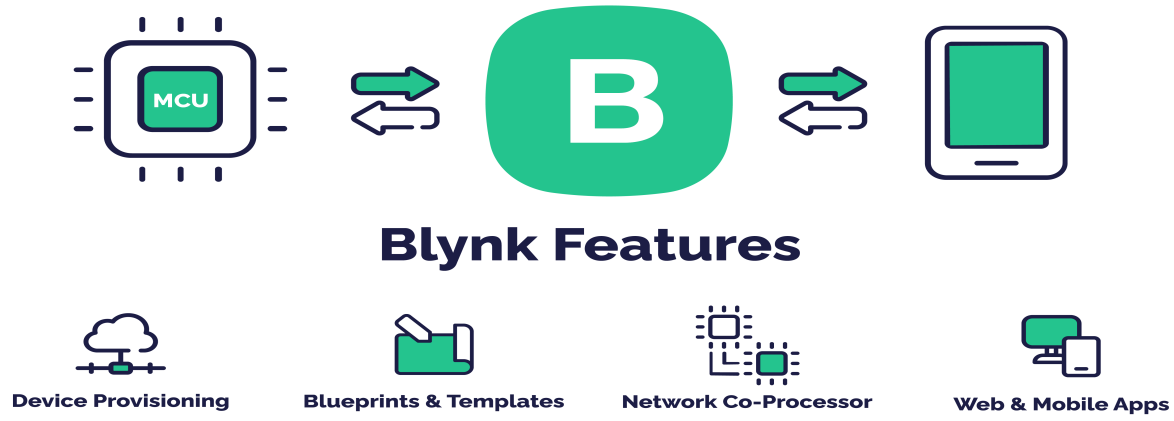


Fig2.0: Blynk

Blynk has been widely adopted in academic IoT projects and industrial prototyping due to its rapid development cycle, cross-platform support, and the availability of a free tier suitable for small-scale projects and academic demonstrations. The platform supports a broad range of hardware including Arduino, ESP8266, ESP32, Raspberry Pi, and commercial development boards, communicating via MQTT or HTTP protocols over Wi-Fi, Ethernet, or cellular connections.

3.13 Blynk Integration in the Fire Detection System

The Blynk IoT platform was integrated into the fire detection system as the primary alert and monitoring interface, replacing the email-only notification approach with a comprehensive

real-time IoT dashboard providing live sensor readings, historical trend graphs, visual alert indicators, and instant mobile push notifications.

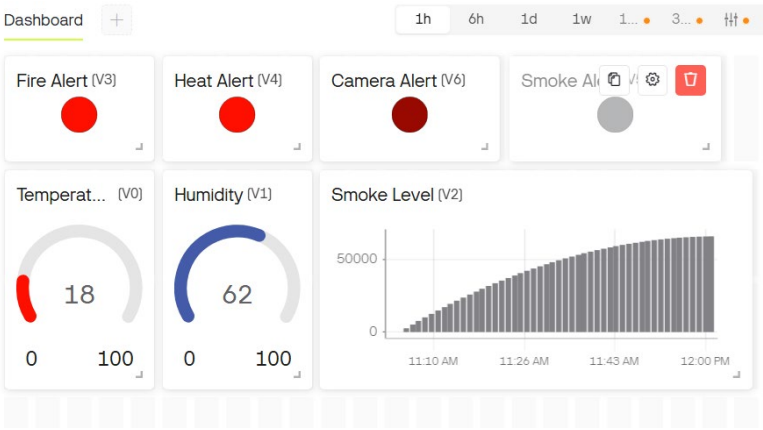


Fig3.0: Blynk dashboard

3.13.1 Template and Device Configuration

The integration was configured through the following steps on the Blynk cloud platform:

- A new Template was created on blynk. cloud with the name Forest Fire Detection System, hardware type set to Raspberry Pi, and connection type set to WIFI.
- Eight Data Streams were defined within the template, each mapped to a virtual pin and configured with appropriate data types and value ranges as detailed in Table 3.8 below.
- A new Device was provisioned from the template, generating a unique 32-character authentication token that was embedded in the system configuration file (config.py) on the Raspberry Pi.
- An Event was configured with the code fire_alert, description Fire Detected by Sensor Fusion System, and priority set to Critical, enabling automatic push notification dispatch on confirmed fire events.
- The mobile dashboard was designed in the Blynk application by adding and configuring widgets bound to the defined virtual pins.

Virtual Pin	Name	Data Type	Description
V0	Temperature	Double (0–100)	Ambient temperature in degrees Celsius from DHT22 sensor, updated every 2 seconds
V1	Humidity	Double (0–100)	Relative humidity percentage from DHT22 sensor, updated every 2 seconds

V2	Smoke Level	Integer (0–65535)	Raw 10-bit ADC reading from MQ135 smoke sensor via MCP3008, normalized to 16-bit scale
V3	Fire Alert	Integer (0 or 1)	System-level fire confirmation flag 1 when at least 2 of 3 sensors confirm fire
V4	Heat Alert	Integer (0 or 1)	DHT22 temperature threshold alert 1 when temperature exceeds 45 degrees Celsius
V5	Smoke Alert	Integer (0 or 1)	MQ135 smoke threshold alert 1 when raw ADC value exceeds calibrated baseline by 30%
V6	Camera Alert	Integer (0 or 1)	YOLOv8 AI camera alert 1 when fire or smoke detected in 4 or more of last 20 frames
V7	Dataset Count	Integer (0–10000)	Running count of labelled training images automatically saved to the dataset directory

Table 3.8: Blynk virtual pin assignments and data descriptions.

3.13.2 Dashboard Widget Configuration

The Blynk application dashboard was configured with the following widgets to provide a comprehensive real-time monitoring interface:

Widget	**Virtual Pin**	**Configuration and Purpose**
Gauge	V0	Displays current temperature from 0 to 100 degrees Celsius with color gradient green for normal, red above threshold
Gauge	V1	Displays current relative humidity from 0 to 100 percent
Super Chart	V2	Historical time-series graph of smoke concentration — allows operator to observe gradual smoke level increases preceding a fire event
Super Chart	V0	Historical time-series graph of temperature — shows temperature elevation trend associated with fire proximity
LED Indicator	V3	Large red LED widget — illuminates when the system fire confirmation flag is active, providing immediate visual alert status

LED Indicator	V4	Orange LED — indicates heat sensor alert status independently
LED Indicator	V5	Yellow LED — indicates smoke sensor alert status independently
LED Indicator	V6	Blue LED — indicates AI camera detection alert status independently
Value Display	V7	Numeric display of the dataset image count — demonstrates the continuous learning data collection capability to reviewers
Notification Widget	Event	Configured to receive push notifications on the fire_alert event — delivers instant mobile alerts with reason string even when the app is closed

Table 3.9: Blynk dashboard widget configuration.

3.13.3 Software Implementation

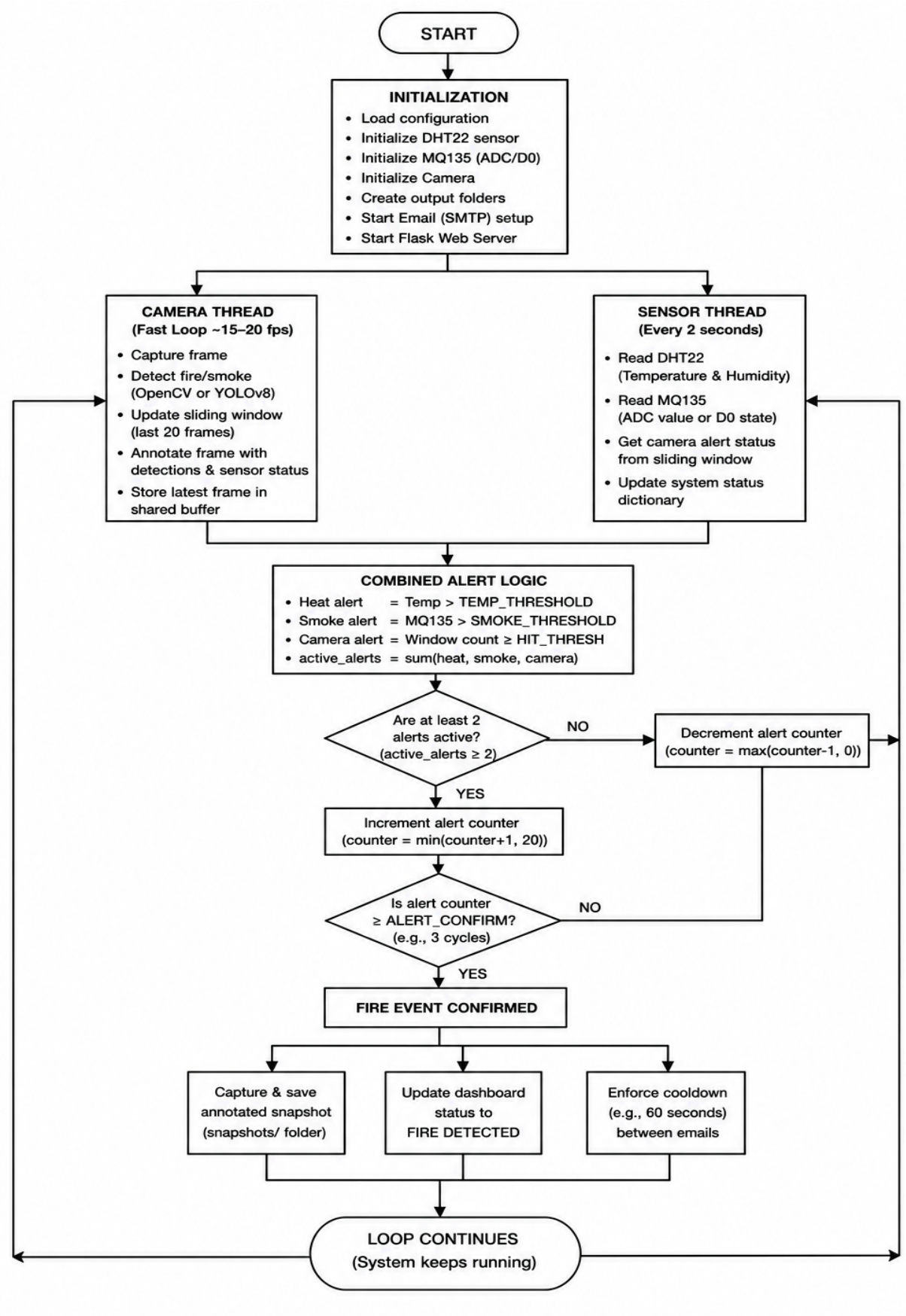
The Blynk integration was implemented as a dedicated module within the alerts/ directory of the AI fire detection system, encapsulating all Blynk communication logic within a BlynkAlert class. The module uses the blynklib Python library, which implements the Blynk protocol over a persistent WebSocket connection to the Blynk cloud server.

The BlynkAlert class exposes two principal methods:

- update (): Called every two seconds from the sensor polling thread, this method writes all eight virtual pin values to the Blynk cloud in a single update cycle. The method also calls blynk.run () to process any incoming messages from the server, maintaining the connection heartbeat and handling server-side commands.
- send_alert (): Called when a fire event is confirmed, this method triggers the fire_alert event on the Blynk platform, which in turn dispatches push notifications to all registered mobile devices. A 30-second cooldown is enforced between successive event triggers to prevent notification flooding during sustained fire events while maintaining timely alerting.

The Blynk update loop runs within the existing sensor polling thread at two-second intervals, adding negligible overhead to the sensor reading and alert evaluation logic. The update call is wrapped in a try-except block to ensure that transient network connectivity issues which may occur during Wi-Fi handoffs or server maintenance do not interrupt the local sensor monitoring and fire detection functions. The system continues to operate fully in offline mode if the Blynk connection is unavailable, with all alert logic, snapshot saving, and local web dashboard streaming continuing unaffected

3.14 Combined Fire Detection Logic Flowchart:



Chapter Four conclusion and recommendations

4.1Conclusion

This project successfully designed, implemented, and tested an intelligent IoT-based system for early forest fire detection and continuous real-time monitoring. Built on a Raspberry Pi 4 Model B single-board computer, the system integrates three independent sensing modalities a DHT22 temperature and humidity sensor, an MQ135 smoke and gas sensor interfaced via an MCP3008 analog-to-digital converter, and a camera module for visual fire detection into a unified detection pipeline governed by a multi-sensor voting mechanism that confirms fire events only when at least two of the three sensing layers agree simultaneously.

The system was implemented in two complementary use cases. The first use case employs classical OpenCV computer vision techniques specifically background subtraction and brightness differential analysis for camera-based fire detection, paired with the digital threshold output of the MQ135 sensor. This approach prioritizes simplicity, low cost, and minimal hardware requirements, making it well suited for budget-conscious deployments. The second use case replaces the classical camera detection with a YOLOv8 Nano deep learning object detection model pre-trained on fire and smoke imagery, and utilizes the full analog output of the MQ135 sensor via the MCP3008 ADC for quantitative smoke concentration measurement. This AI-enhanced variant achieves substantially higher camera-based detection accuracy, the ability to simultaneously classify both fire and smoke, and an automatic dataset collection capability that continuously saves labelled training images for future model improvement.

Both use cases are integrated with the Blynk IoT cloud platform via its HTTP REST API, providing a professional real-time monitoring dashboard accessible from any device on the network. The dashboard displays live temperature, humidity, and smoke concentration readings updated every two seconds, individual alert status indicators for each sensing modality, and a historical time-series graph of sensor measurements. When a fire event is confirmed, the system dispatches an instant push notification to the operator's mobile device via the Blynk event system, and simultaneously saves a timestamped snapshot image annotated with detection bounding boxes to local storage.

The two-use-case architecture adopted in this project serves both academic and practical purposes. Academically, it demonstrates a clear understanding of the trade-off spectrum between system complexity and detection capability, and provides a compelling basis for comparative analysis between classical computer vision and modern deep learning approaches in the context of real-time embedded fire detection. Practically, it offers a deployment-ready baseline system for cost-sensitive applications and a higher-capability AI system for demanding outdoor environments such as the aerial drone-mounted monitoring scenario that motivated the project.

A key limitation acknowledged in this work is that camera-based fire detection using either approach is most effective in outdoor environments with a stable, predominantly dark and green background the typical appearance of forested terrain from an aerial perspective. Indoor testing with warm artificial lighting presents significant challenges for background subtraction and color-based detection methods. The YOLOv8 model,

being trained on diverse fire imagery, handles variable lighting conditions more robustly than the classical approach, which reinforces the appropriateness of Use Case 2 for the intended drone-mounted deployment scenario.

4.2Recommendations

Based on the experience gained during the design and implementation of this system, and the limitations identified during testing, the following recommendations are proposed for future development iterations:

- **Drone Integration for Aerial Deployment**

The system was designed with drone-mounted deployment as the primary intended use case, but was tested in a stationary ground-level configuration due to the unavailability of a drone platform during the project period. Integrating the Raspberry Pi detection unit onto an autonomous unmanned aerial vehicle (UAV) would enable wide-area forest monitoring coverage, access to remote and otherwise inaccessible terrain, and a bird's-eye camera perspective that significantly improves fire and smoke detection accuracy. The predominantly green and dark forest canopy background visible from aerial altitude would substantially improve the performance of both the OpenCV background subtraction method and the YOLOv8 model, both of which were shown to perform more reliably against uniform dark backgrounds. Future work should include UAV integration testing with the detection unit mounted on a stabilized gimbal to reduce motion blur and vibration artefacts in the camera feed.

- **Model Fine-Tuning on Collected Dataset**

The AI-enhanced system automatically collects labelled training images in YOLO format during every fire detection event, building a dataset of imagery captured by the actual deployment camera under real operating conditions. This dataset represents a valuable resource for domain adaptation fine-tuning the pre-trained YOLOv8 model on images from the specific camera, lighting conditions, and fire types encountered in the deployment environment. Future work should implement a periodic offline retraining pipeline in which the accumulated dataset is used to update the model weights, progressively improving detection accuracy for the specific deployment context over time. This self-improving capability, demonstrated conceptually in this project through the dataset collection mechanism, represents one of the most academically significant aspects of the AI-enhanced use case.

- **Fire Suppression Integration**

An early-detection system is most valuable when paired with an early-response capability. Future iterations could integrate a fire suppression mechanism such as a fire extinguishing ball holder or a pressurized retardant dispenser mounted on the drone that can be triggered autonomously upon fire confirmation. For ground-based fixed installations, a relay-controlled water spray or retardant release mechanism could be activated by the Raspberry Pi GPIO upon confirmed detection, enabling automatic suppression of small fires before they spread. The multi-sensor voting logic already implemented in this system provides a robust confirmation mechanism that would minimize the risk of accidental suppression activation from false positive detections.

- **Additional Environmental Sensors**

The current sensor suite focuses on temperature, humidity, and smoke concentration. Adding complementary sensors would improve both detection accuracy and the richness of the data available for post-incident analysis:

  - Wind Speed and Direction Sensor (anemometer): Wind is the primary driver of fire spread rate and direction. Integrating a wind sensor would allow the system to predict the likely spread trajectory of a detected fire and priorities alert urgency accordingly.
  - Air Quality Sensor (particulate matter PM2.5/PM10): Sensors such as the PMS5003 measure fine particulate matter concentrations, which are a more specific indicator of combustion smoke than the broad gas sensitivity of the MQ135. Adding particulate measurement would reduce false positives from non-combustion gases such as vehicle exhaust or agricultural chemicals.
  - Carbon Monoxide Sensor (MQ-7): CO is a primary product of incomplete combustion and a reliable early indicator of smoldering fires that may not yet be producing visible smoke or significant heat. Adding CO detection as a fourth sensing modality would extend the system's ability to detect fires in their very earliest stages.
  - Infrared Thermometer (MLX90614): A non-contact infrared temperature sensor can detect the radiated heat signature of a fire from a greater distance than the DHT22 contact thermistor, and is not affected by ambient air temperature fluctuations caused by sunlight or weather.

- **Thermal Camera**

The Raspberry Pi Camera Module v1.3 used in this project operates in the visible light spectrum and is therefore limited to daytime monitoring conditions. A significantly more capable alternative for forest fire detection is the integration of a dedicated thermal infrared camera, such as the FLIR Lepton or the MLX90640 thermal array sensor. Unlike visible light cameras, thermal cameras detect radiated heat energy rather than reflected light, making them completely independent of ambient lighting conditions they operate equally well in full daylight, complete darkness, and through light smoke obscuration.

References:

[1] A Review on Forest Fire Detection Techniques: Past, Present, and Sustainable Future by Juan-Carlos Jiménez-Muñoz.

[2] Techniques for Wildfire Detection and Monitoring
NASA Earthdata.

[3] Raspberry Pi Foundation (2019). Raspberry Pi 4 Model B: Datasheet. raspberrypi.com.

[4] OmniVision Technologies (2012). OV5647 Color CMOS 5-Megapixel (2592×1944) CameraChip™ Sensor. OmniVision Datasheet.

[5] Khanna, S. et al. (2020). Measurement of CO_2 Gas Concentration Using MQ135 Sensor for Air Pollution Monitoring. International Journal of Engineering Research and Technology, 13(10), 2955–2959. ResearchGate.

[6] Aosong Electronics Co. (2010). AM2302 Product Manual (DHT22). aosong.com.

[7] Kester, W. (2005). Data Conversion Handbook. Newnes/Analog Devices.

[8] Gottuk, D.T. et al. (2002). Investigation of Multi-Sensor Algorithms for Fire Detection. Fire Technology, 38, 183–208. doi:10.1023/A:1025378100781 Springer.

[9] Chen, T.H., Kao, C.L., Chang, S.M. (2003). An Intelligent Real-Time Fire-Detection Method Based on Video Processing. Proceedings of IEEE 37th International Carnahan Conference on Security Technology, 104–111. Utah State University.

[10] Hassan, M. et al. (2025). Edge-Based Autonomous Fire and Smoke Detection Using MobileNetV2. PMC12567645 nih.

Appendix: A

Main.py

Importing & initialization

```
import cv2
import threading
import time
import os
from datetime import datetime
from config import (
    SMOKE_THRESHOLD, TEMP_THRESHOLD, HUMIDITY_DROP,
    ALERT_CONFIRM, EMAIL_COOLDOWN, SNAPSHOT_DIR,
    BLYNK_TOKEN
)
from sensors.dht22        import DHT22Sensor
from sensors.mq135        import MQ135Sensor
from sensors.camera       import CameraDetector
from alerts.email_alert   import EmailAlert
from alerts.blynk_alert   import BlynkAlert
from web.stream           import create_stream
os.makedirs(SNAPSHOT_DIR, exist_ok=True)
print("=" * 55)
print("  Forest Fire Detection System — AI Enhanced")
print("  YOLOv8 Nano + DHT22 + MQ135 + Blynk IoT")
print("  Graduation Project")
print("=" * 55)
```

```
dht    = DHT22Sensor()
mq135  = MQ135Sensor()
camera = CameraDetector(
    model_path  = "models/fire_model.pt",
    fire_conf   = 0.20,
    imgsz       = 320,
    hit_thresh  = 4,
    dataset_dir = "dataset"
)
blynk = BlynkAlert(BLYNK_TOKEN)
lock         = threading.Lock()
latest_frame = None
status = {
    "fire":          False,
    "heat":          False,
    "smoke":         False,
    "camera":        False,
    "temperature":   0.0,
    "humidity":      0.0,
    "smoke_raw":     0,
    "alert_count":   0,
    "dataset_count": 0,
    "detections":    [],
}
```

Camera & sensors Loops:

```
last_email    = 0
last_blynk    = 0
alert_counter = 0


def save_snapshot(frame):
    ts       = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{SNAPSHOT_DIR}/fire_{ts}.jpg"
    cv2.imwrite(filename, cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))
    print(f"Snapshot saved: {filename}")
    return filename


# ── Camera loop — fast ~20fps ──
def camera_loop():
    global latest_frame
    while True:
        try:
            fire_area, fire_regions, display = camera.read()


            if display is None:
                time.sleep(0.05)
                continue
            with lock:
                s = dict(status)
```

```
label = "*** FIRE DETECTED ***" if s['fire'] else "Monitoring..."
        color = (255, 50, 50) if s['fire'] else (50, 220, 50)
        cv2.putText(display, label, (10, 35),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, color, 2)
        cv2.putText(display,
            f"Temp:{s['temperature']:.1f}C  "
            f"Hum:{s['humidity']:.1f}%  "
            f"Smoke:{s['smoke_raw']}",
            (10, 65),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
        cv2.putText(display,
            f"Heat:{'!' if s['heat'] else 'OK'}  "
            f"Smoke:{'!' if s['smoke'] else 'OK'}  "
            f"AI:{'!' if s['camera'] else 'OK'}  "
            f"Alert:{s['alert_count']}/20",
            (10, 88),
            cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)
        cv2.putText(display,
            f"Dataset: {s['dataset_count']} images",
            (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 0.45, (100, 220, 100), 1)
        cv2.putText(display,
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            (10, 468),
            cv2.FONT_HERSHEY_SIMPLEX, 0.42, (160, 160, 160), 1)
```

```
            with lock:
                latest_frame = display

        except Exception as e:
            print(f"Camera loop error: {e}")
        time.sleep(0.05)

# ── Sensor loop — every 2 seconds ──
def sensor_loop():
    global last_email, last_blynk, alert_counter
    while True:
        try:
            temp, humidity, _ = dht.read()
            smoke_raw, _, _   = mq135.read()

            heat_alert    = dht.is_heat_alert(TEMP_THRESHOLD)
            smoke_alert   = mq135.is_smoke_alert(SMOKE_THRESHOLD)
            camera_alert  = camera.is_fire_alert()
            detections    = camera.get_detections_str()
            dataset_count = camera.get_dataset_count()

            # 2 of 3 voting
            alerts = sum([heat_alert, smoke_alert, camera_alert])
            if alerts >= 2:
                alert_counter = min(alert_counter + 1, 20)

                alert  counter = max(alert  counter - 1, 0)
```

Alert & Blynk Dashboard:

```
        else:
            alert_counter = max (alert_counter - 1, 0)
        fire_confirmed = alert_counter >= ALERT_CONFIRM
        # ── Send Blynk alert notification ──
        if fire_confirmed:
            now = time.time()
            if now - last_blynk > 30:  # notify max every 30 seconds
                reasons = []
                if heat_alert:   reasons.append(f"Temp {temp:.1f}C")
                if smoke_alert:  reasons.append(f"Smoke {smoke_raw}")
                if camera_alert: reasons.append("Visual fire detected")
                blynk.send_alert(
                    f"FIRE ALERT! {' + '.join(reasons)}"
                )
                last_blynk = now
        # ── Update Blynk dashboard every 2 seconds ──
        blynk.update(
            temperature   = temp,
            humidity      = humidity,
            smoke_raw     = smoke_raw,
            fire_alert    = fire_confirmed,
            heat_alert    = heat_alert,
            smoke_alert   = smoke_alert,
            camera_alert  = camera_alert,

        )
```

```
# Update shared status
with lock:
    status.update({
        "fire":          fire_confirmed,
        "heat":          heat_alert,
        "smoke":         smoke_alert,
        "camera":        camera_alert,
        "temperature":   temp,
        "humidity":      humidity,
        "smoke_raw":     smoke_raw,
        "alert_count":   alert_counter,
        "dataset_count": dataset_count,
        "detections":    detections,
    })

print(
    f"[{datetime.now().strftime('%H:%M:%S')}]  "
    f"Temp:{temp:.1f}C  Hum:{humidity:.1f}%  "
    f"Smoke:{smoke_raw}  "
    f"Heat:{'!' if heat_alert else 'OK'}  "
    f"Smoke:{'!' if smoke_alert else 'OK'}  "
    f"AI:{'!' if camera_alert else 'OK'}  "
    f"Fire:{fire_confirmed}  "
    f"Dataset:{dataset_count}imgs"
)
```

Frame Generator:

```
        except Exception as e:
            print(f"Sensor loop error: {e}")
        time.sleep(2)


# ── Frame generator ──
def get_frames():
    while True:
        with lock:
            frame = latest_frame
        if frame is None:
            time.sleep(0.05)
            continue
        _, buffer = cv2.imencode(
            '.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 50]
        )
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' +
               buffer.tobytes() + b'\r\n')
        time.sleep(0.05)


def get_status():
    with lock:
        return dict(status)
```

Main function:

```
if __name__ == '__main__':
    threading.Thread(target=camera_loop, daemon=True).start()
    threading.Thread(target=sensor_loop, daemon=True).start()

    print("\nSystem running!")
    print("Open stream: http://raspberrypi.local:5000")
    print("Open Blynk app on your phone\n")

    flask_app = create_stream(get_frames, get_status)
    flask_app.run(host='0.0.0.0', port=5000, threaded=True)
```